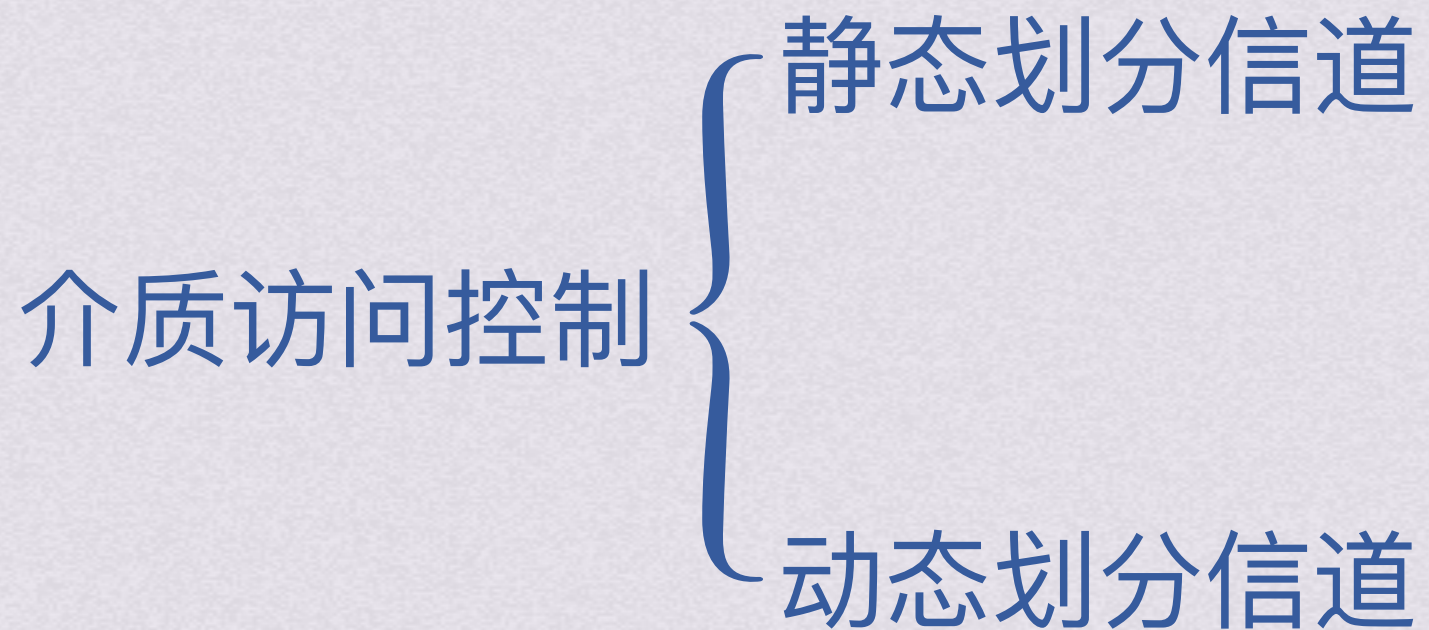
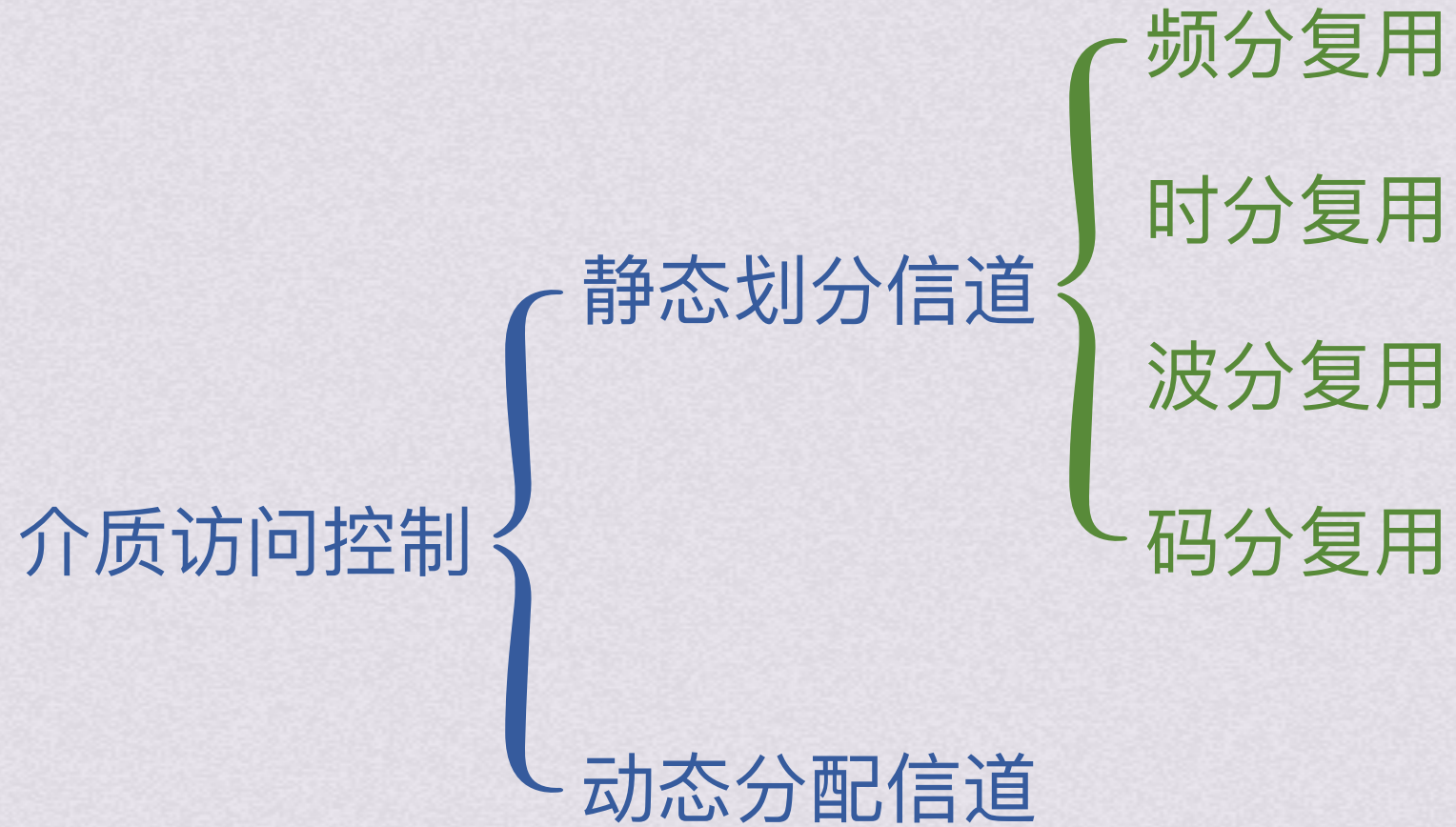


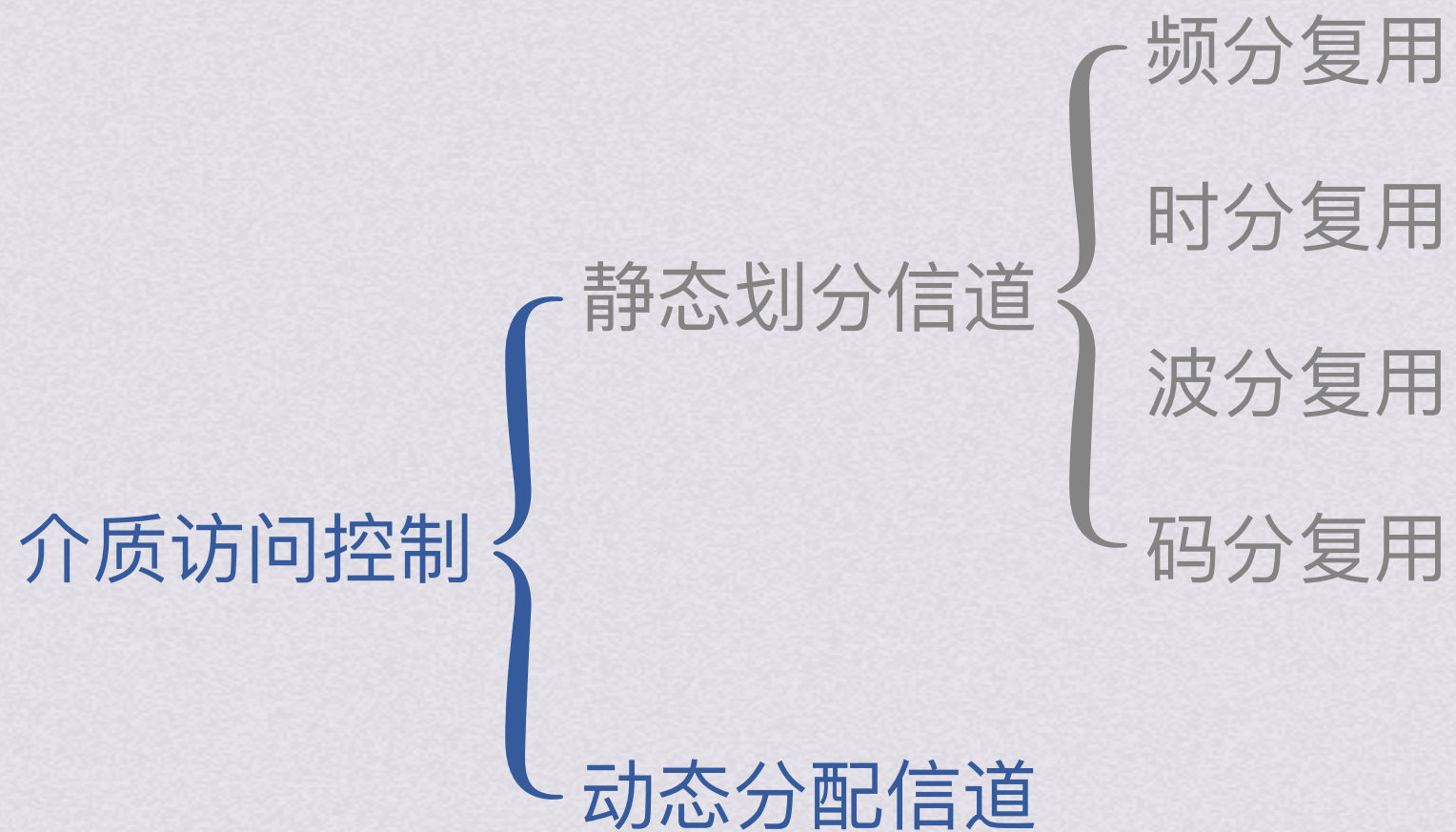
bok25

基
礎

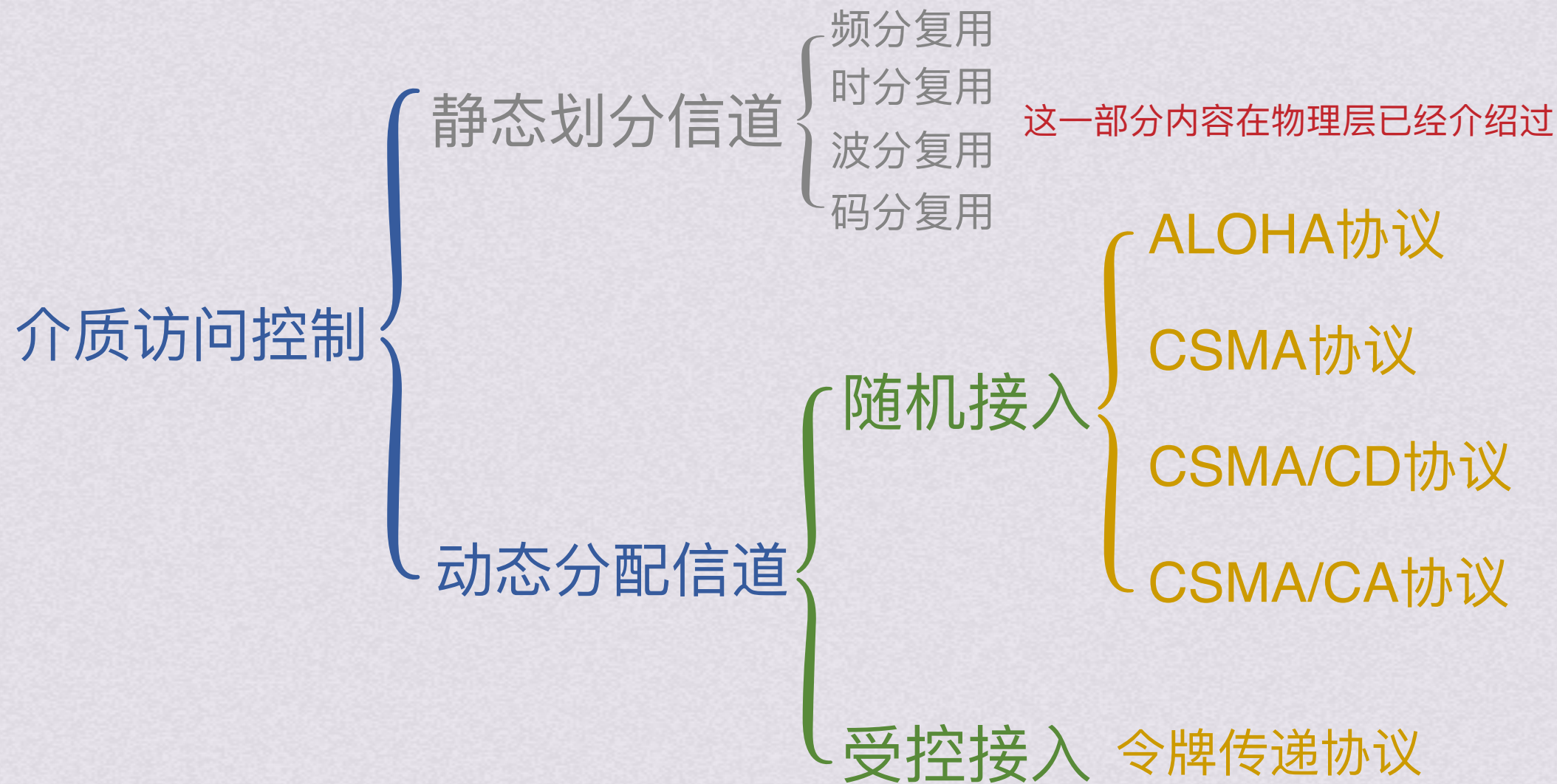
課、計算機網絡







这一部分内容在物理层已经介绍过





随机接入

ALOHA协议 纯ALOHA协议

总线形网络中任何站点需要发送数据时，可以不进行任何检测就发送数据。若在一段时间内未收到确认，就认为传输过程中发生冲突，发送站点等待一段时间后在发送数据，直到发送成功

假设B给C发消息的情况



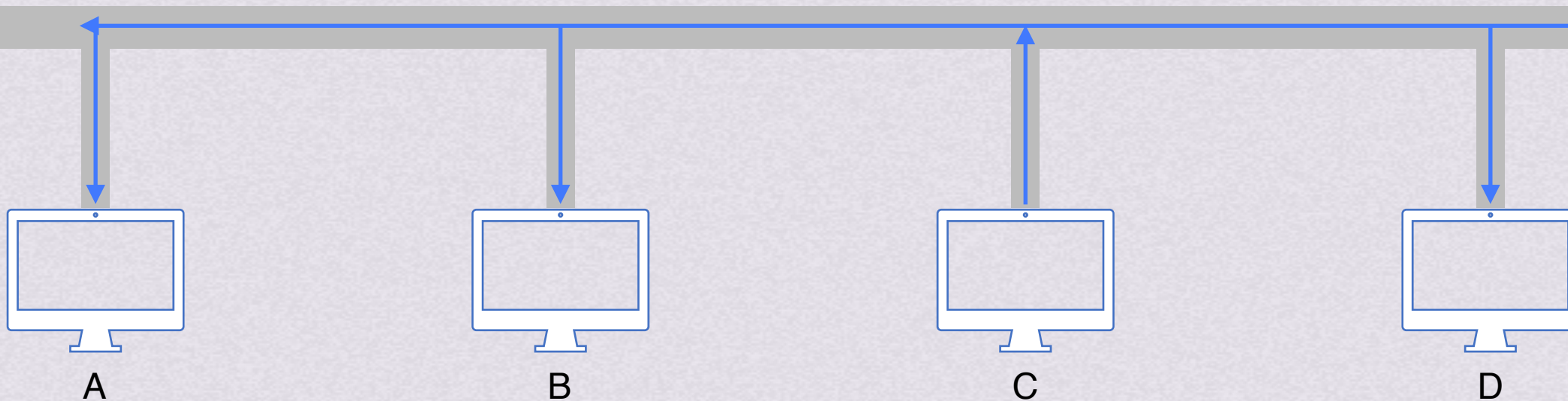


随机接入

ALOHA协议 纯ALOHA协议

总线形网络中任何站点需要发送数据时，可以不进行任何检测就发送数据。若在一段时间内未收到确认，就认为传输过程中发生冲突，发送站点等待一段时间后在发送数据，直到发送成功

假设B给C发消息的情况
数据和确认都发送成功





随机接入

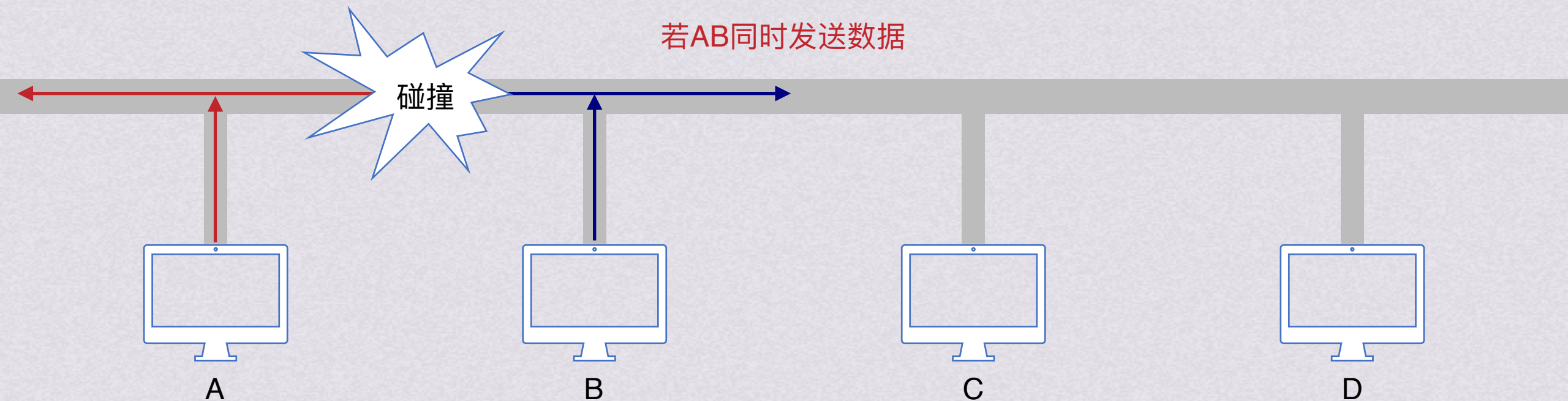
ALOHA协议 纯ALOHA协议

总线形网络中任何站点需要发送数据时，可以不进行任何检测就发送数据。若在一段时间内未收到确认，就认为传输过程中发生冲突，发送站点等待一段时间后在发送数据，直到发送成功

假设B给C发消息的情况

数据和确认都发送成功

若AB同时发送数据





随机接入

ALOHA协议 纯ALOHA协议

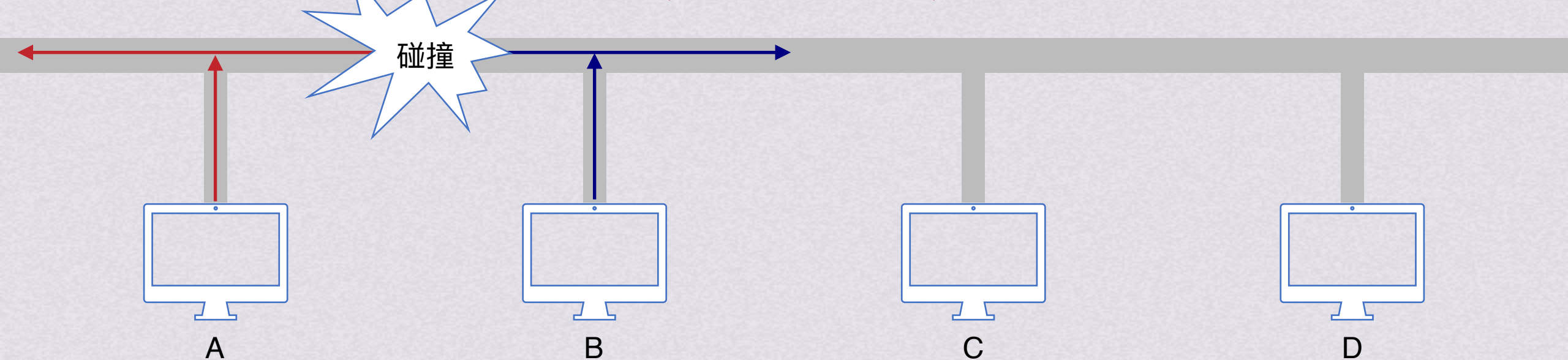
总线形网络中任何站点需要发送数据时，可以不进行任何检测就发送数据。若在一段时间内未收到确认，就认为传输过程中发生冲突，发送站点等待一段时间后在发送数据，直到发送成功

若AB同时发送数据

AB都不会收到确认，因此他们会在随机一段时间后重新发送数据

因为时间随机，所以下次发送可能就不会碰撞了

纯ALOHA网络的吞吐量很低，为了克服这个缺点，产生了时隙ALOHA协议

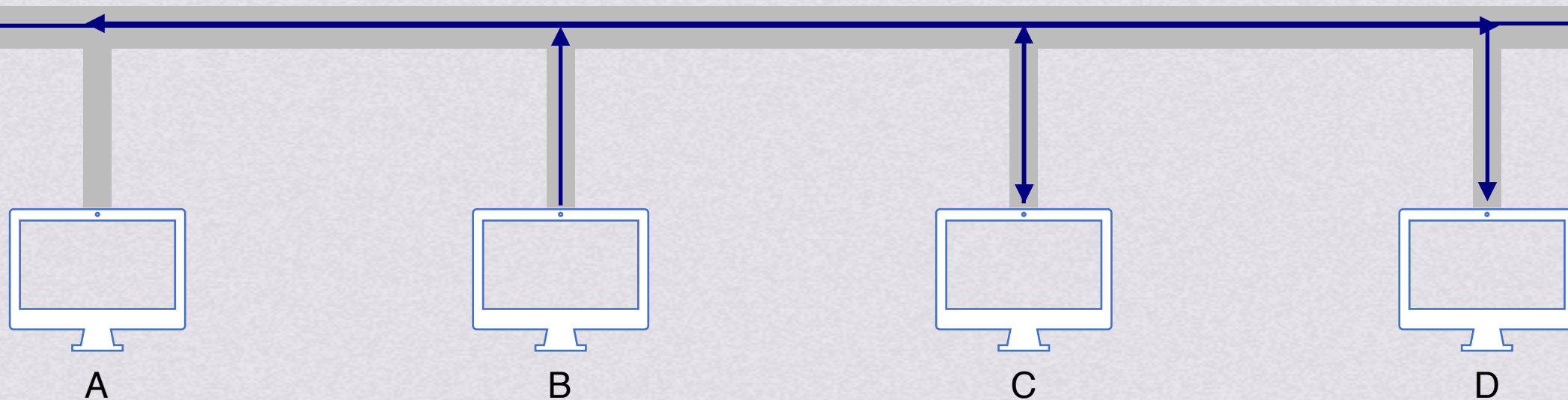




随机接入

ALOHA协议 时隙ALOHA协议

同步各站点的时间，将时间划分为一段段等长的时隙，规定站点只能在每个时隙开始时才能发送帧，发送一帧时间必须小于等于时隙长度，这样就能降低产生冲突的可能性。冲突后重传策略与纯ALOHA协议相似





随机接入



CS：载波监听，也就是在发送帧之前先检查总线上是否有其他站点在发送数据

MA：多址接入，多个站连接在一条总线上，竞争使用总线

CSMA协议，Carrier Sense Multiple Access，即载波监听多路访问。它通过载波监听装置实现了对信道的监听，发现信道空闲后再发送，就可以降低冲突的概率



随机接入

CSMA协议

CS：载波监听，也就是在发送帧之前先检查总线上是否有其他站点在发送数据

MA：多址接入，多个站连接在一条总线上，竞争使用总线

CSMA协议，Carrier Sense Multiple Access，即载波监听多路访问。它通过载波监听装置实现了对信道的监听，发现信道空闲后再发送，就可以降低冲突的概率

1-坚持CSMA

1-坚持CSMA就是只要监听到信道空闲，就以1(100%)的概率立即发送帧，若监听到信号忙则等待信道空闲

非坚持CSMA

站点要发送数据时，先监听信道，若信道空闲则立刻发送；若信道忙则放弃坚持，等待随机时间后再监听

p-坚持CSMA

仅适用于时分信道，站点要发数据时，监听信道，若信道忙则等到下一个时隙再监听，直到信道空闲；若信道空闲，则以概率p发送数据，以概率1-p推迟到下一个时隙再继续监听，直到数据发送成功



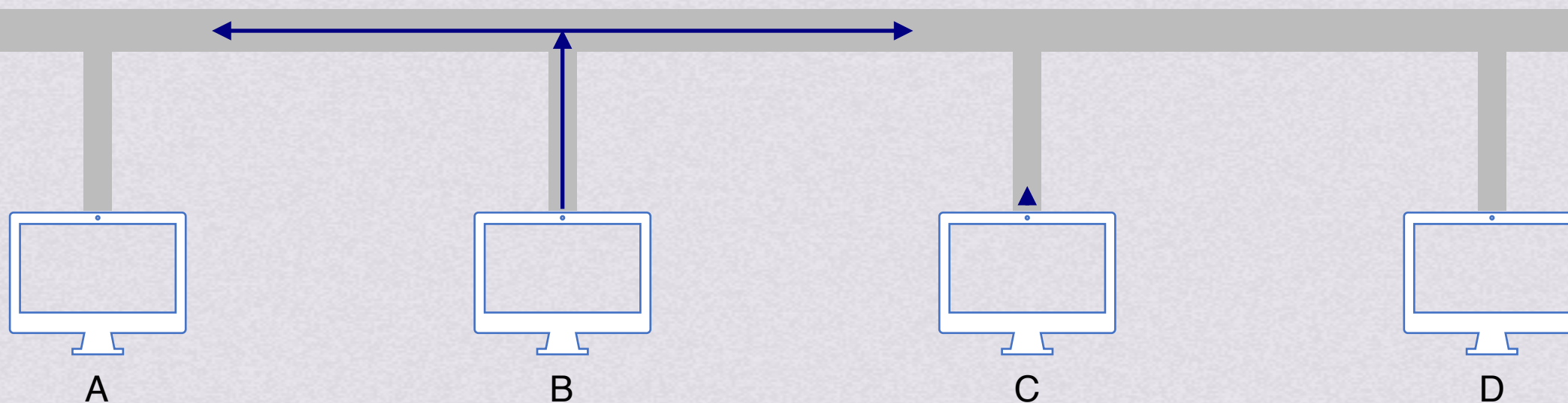
随机接入

CSMA/CD协议

CS: 载波监听, 也就是在发送帧之前先检查总线上是否有其他站点在发送数据

MA: 多址接入, 多个站连接在一条总线上, 竞争使用总线

CD: 碰撞检测, 每一个正在发送帧的站都边发送边检测是否发生碰撞, 一旦发生碰撞, 立刻停止发送, 退避随机时间后再发送



B在发送数据的时候边发送边检测是否发生碰撞



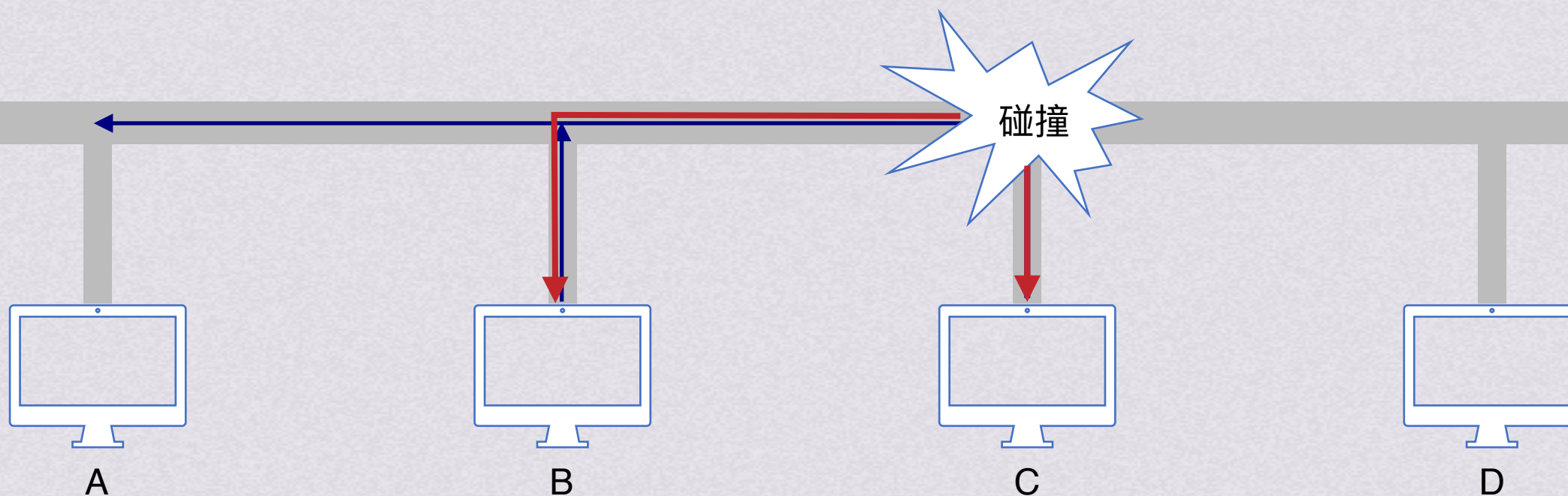
随机接入

CSMA/CD协议

CS: 载波监听, 也就是在发送帧之前先检查总线上是否有其他站点在发送数据

MA: 多址接入, 多个站连接在一条总线上, 竞争使用总线

CD: 碰撞检测, 每一个正在发送帧的站都边发送边检测是否发生碰撞, 一旦发生碰撞, 立刻停止发送, 退避随机时间后再发送



B在发送数据的时候边发送边检测是否发生碰撞 C也会边发边检测是否碰撞

若发送过程中发生碰撞, 第一时间BC并不知晓, 等待碰撞信号传回时BC才知道



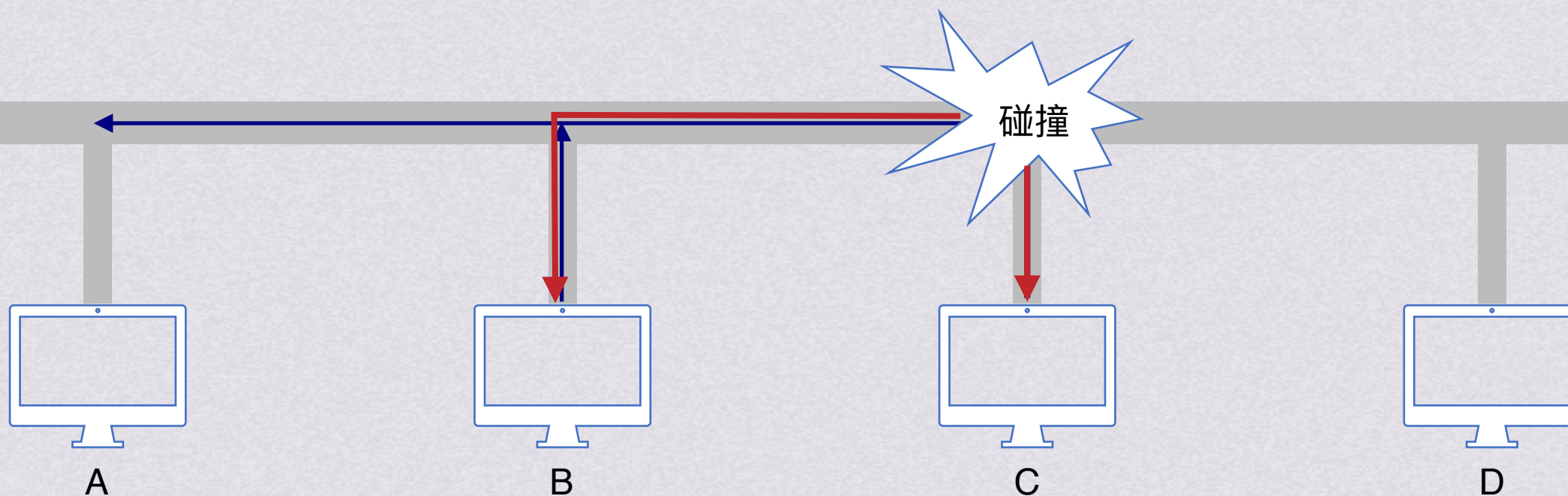
随机接入

CSMA/CD协议

CS：载波监听，也就是在发送帧之前先检查总线上是否有其他站点在发送数据

MA：多址接入，多个站连接在一条总线上，竞争使用总线

CD:碰撞检测，每一个正在发送帧的站都边发送边检测是否发生碰撞，一旦发生碰撞，立刻停止发送，退避随机时间后再发送



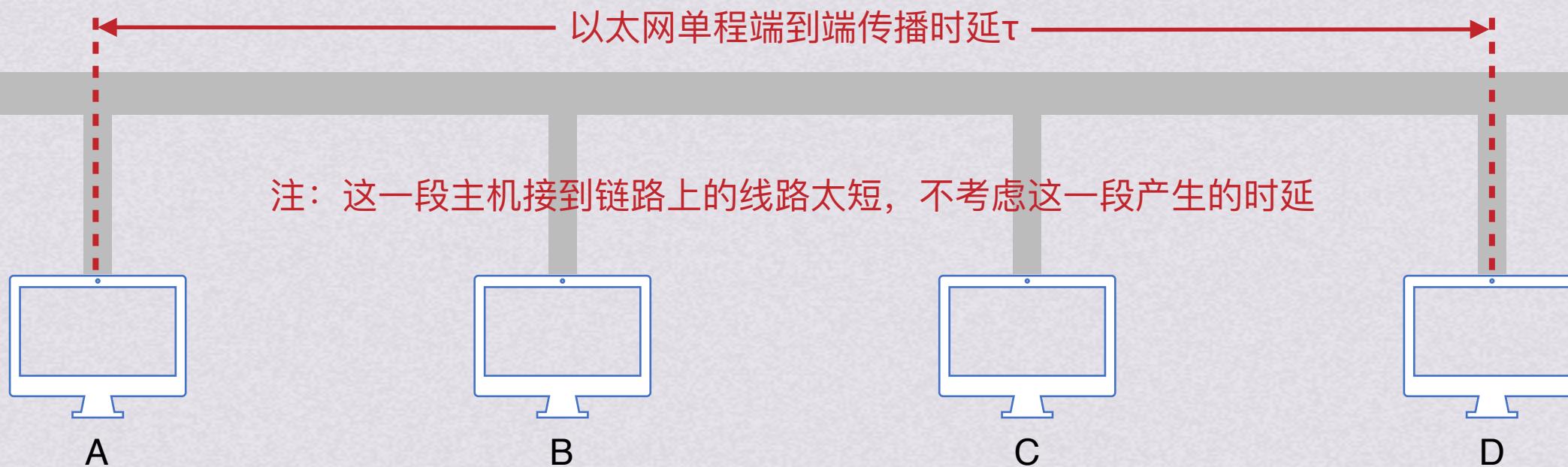
若发送过程中发生碰撞，第一时间BC并不知晓，等待碰撞信号传回时BC才知道
检测到碰撞后，CB各自先后退避一段时间后，重新发送之前发送的帧



随机接入

CSMA/CD协议

争用期(碰撞窗口) τ : 以太网单程端到端传播时延



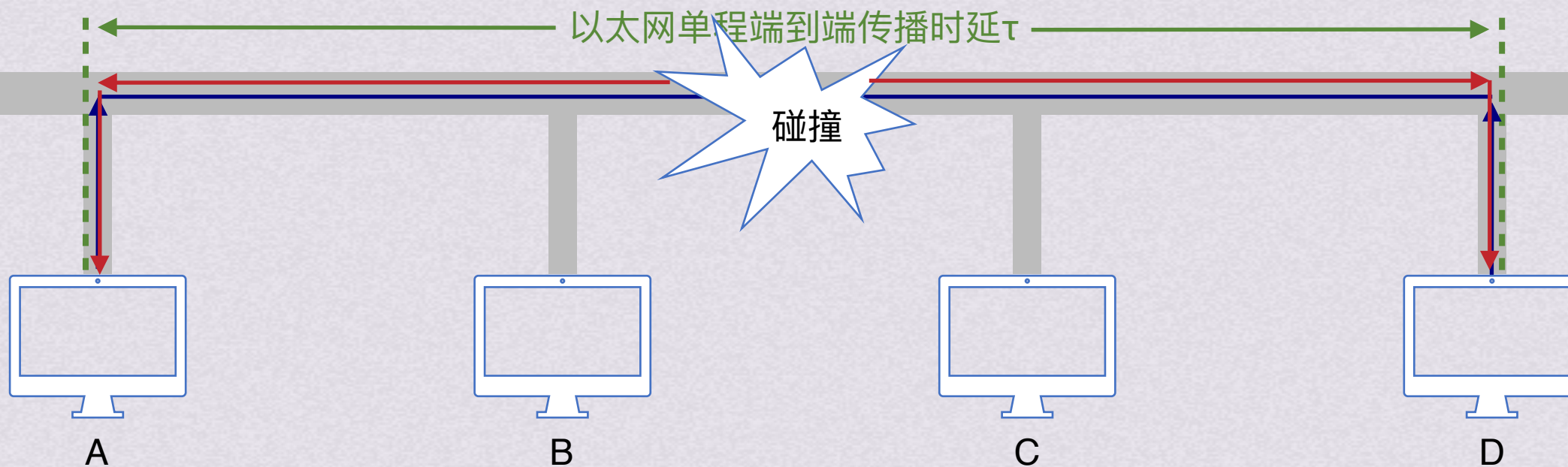


随机接入

CSMA/CD协议

争用期(碰撞窗口) τ : 以太网单程端到端传播时延

假设主机A和D此时都有数据需要发送
因为他们都检测到此时线路是空闲状态，所以都可以发送数据，数据会同时被发出



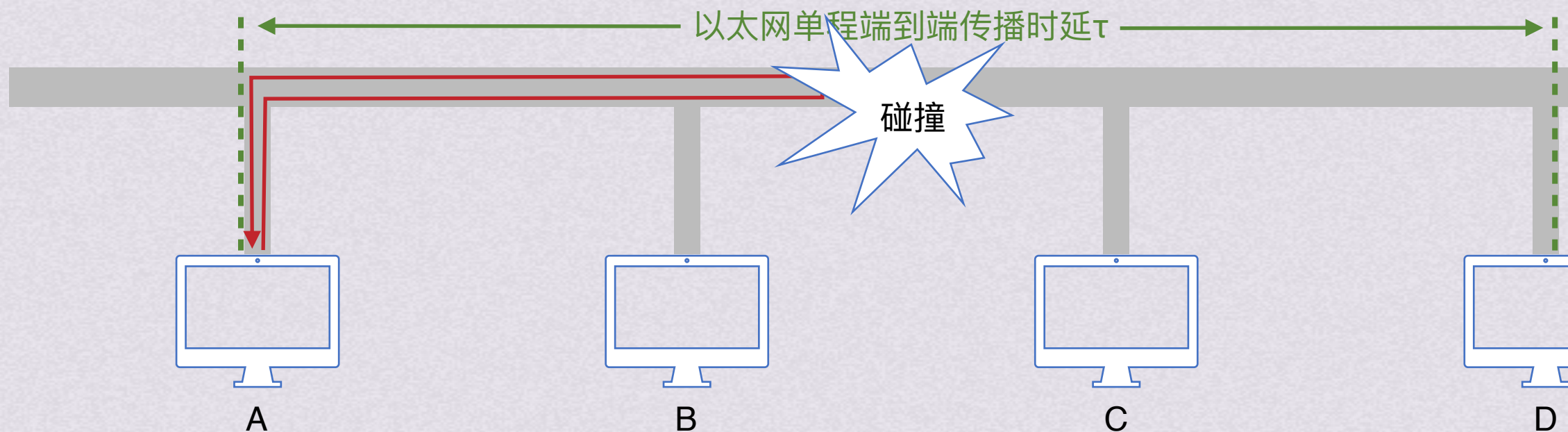


随机接入

CSMA/CD协议

争用期(碰撞窗口) τ : 以太网单程端到端传播时延

假设主机A和D此时都有数据需要发送
因为他们都检测到此时线路是空闲状态, 所以都可以发送数据, 数据会同时被发出



可以观察到A和D都是经过了 τ 时间就能检测到自己发出去的数据发生了碰撞
这是最短情况

我们还可以试着让D发送数据更迟一些

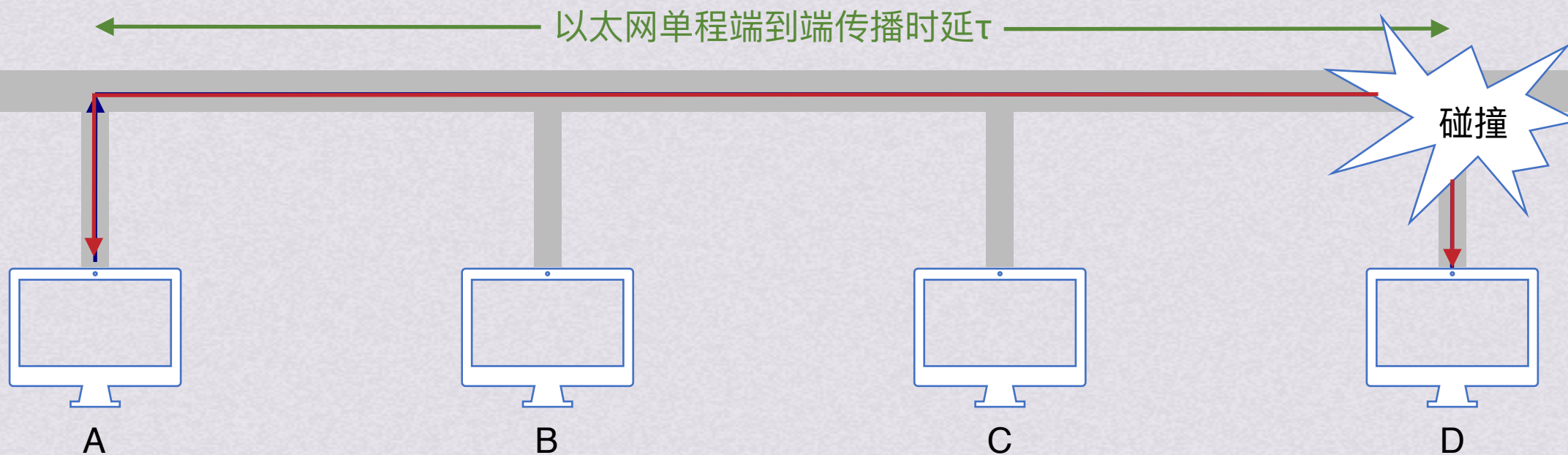


随机接入

CSMA/CD协议

争用期(碰撞窗口) τ : 以太网单程端到端传播时延

假设D在A发送的数据即将到达D的时候才发送数据



我们再来仔细观察一下A发出数据的情况

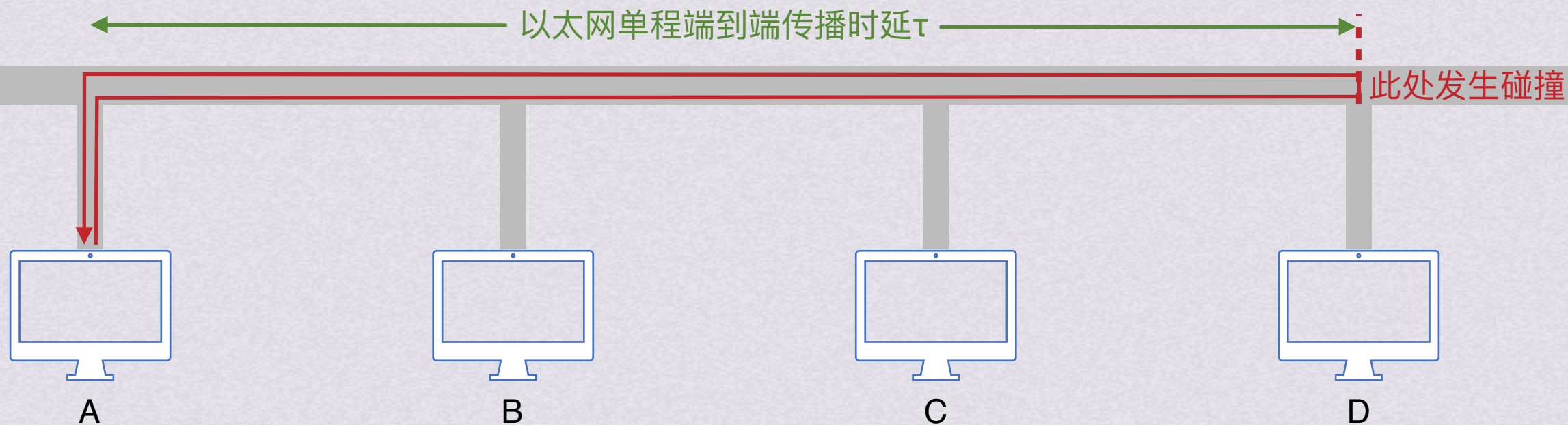


随机接入

CSMA/CD协议

争用期(碰撞窗口) τ : 以太网单程端到端传播时延

假设D在A发送的数据即将到达D的时候才发送数据



我们再来仔细观察一下A发出数据的情况

最坏情况下, A需要一趟最远往返, 也就是 2τ 的时间才能检测到自己发出的数据经过了碰撞

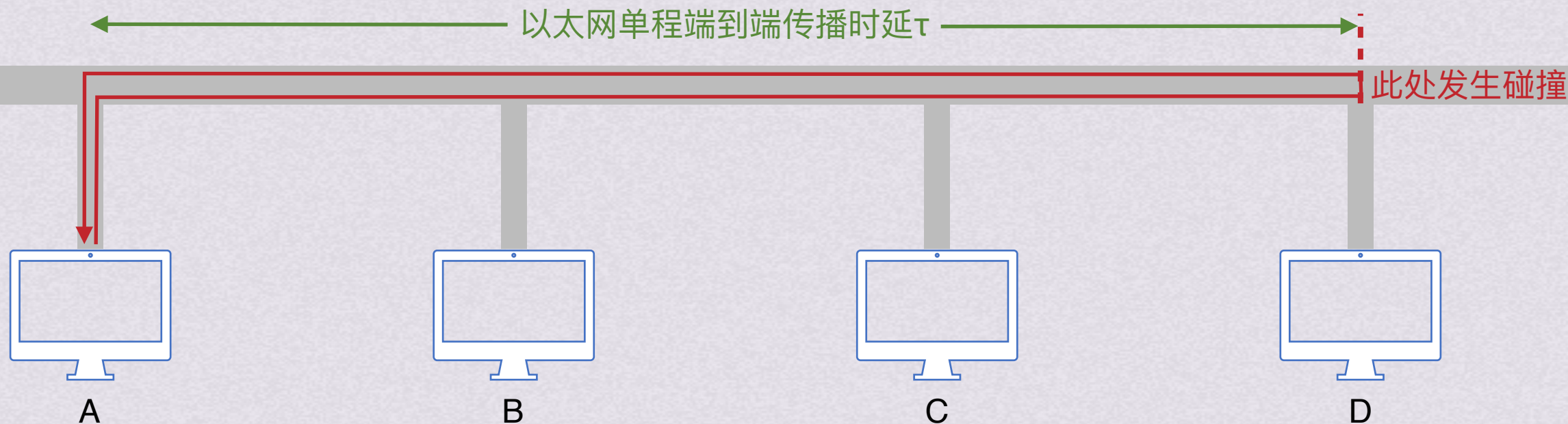


随机接入

CSMA/CD协议

争用期(碰撞窗口) τ : 以太网单程端到端传播时延

假设D在A发送的数据即将到达D的时候才发送数据



我们再来仔细观察一下A发出数据的情况

最坏情况下，A需要一趟最远往返，也就是 2τ 的时间才能检测到自己发出的数据经过了碰撞

所以只有经过争用期这段时间还未检测到碰撞时，才能确认这次传送不会产生碰撞

并且持续发送数据的时间一定要 $\geq 2\tau$

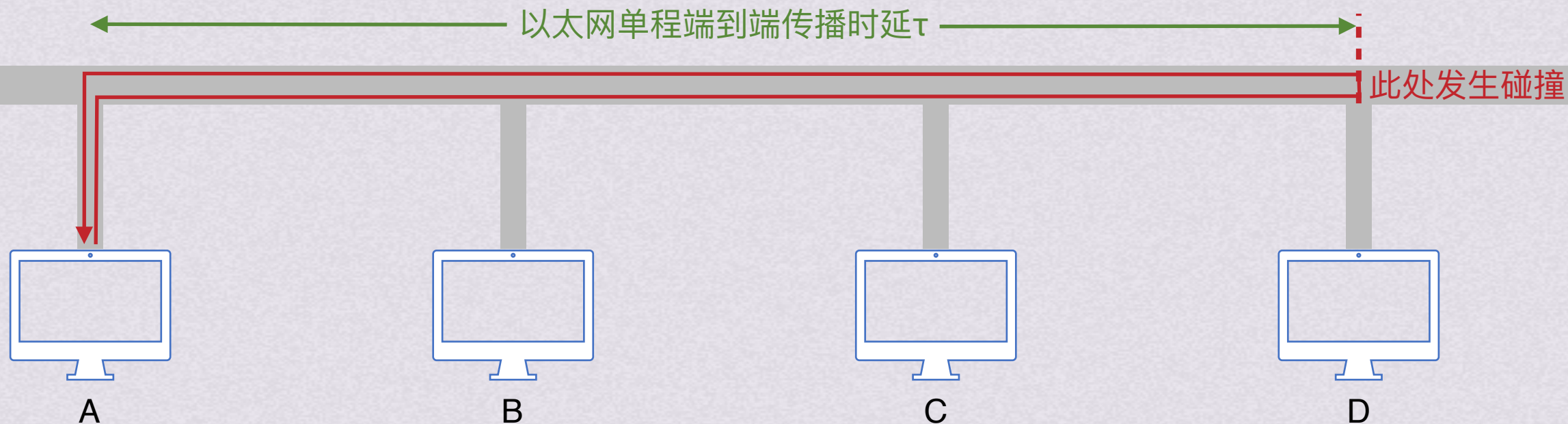


随机接入

CSMA/CD协议

争用期(碰撞窗口) τ : 以太网单程端到端传播时延

假设D在A发送的数据即将到达D的时候才发送数据



我们再来仔细观察一下A发出数据的情况

最坏情况下，A需要一趟最远往返，也就是 2τ 的时间才能检测到自己发出的数据经过了碰撞

前面提到，碰撞检测是只有在发送的时候才会检测的，那么如果我们在最坏的 2τ 时间之前就停止了发送数据

数据一旦发生碰撞我们就无法检测到

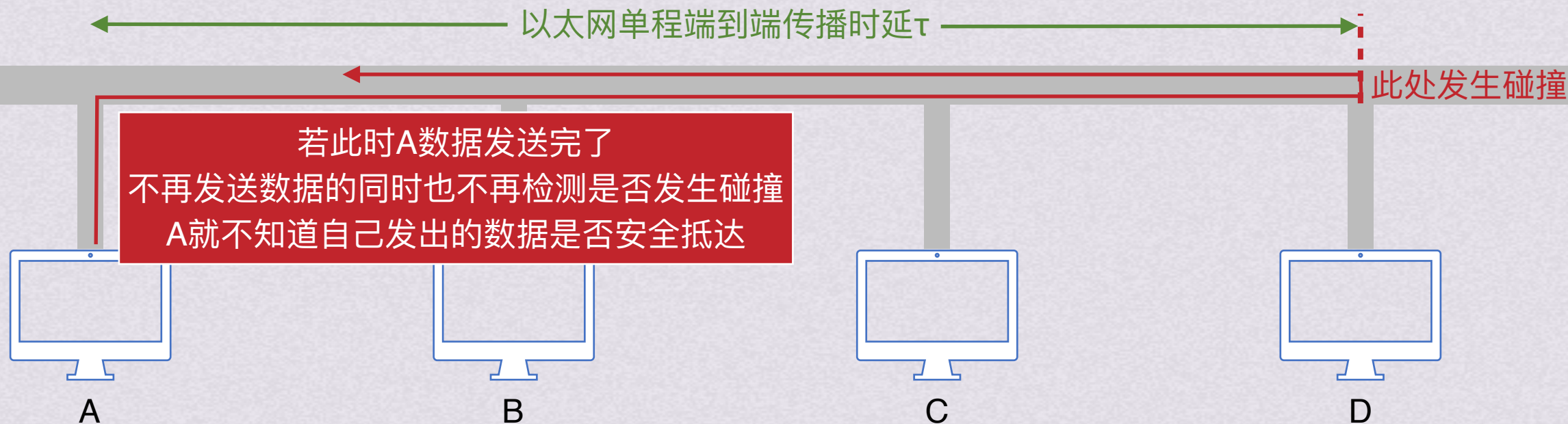


随机接入

CSMA/CD协议

争用期(碰撞窗口) τ : 以太网单程端到端传播时延

假设D在A发送的数据即将到达D的时候才发送数据



我们再来仔细观察一下A发出数据的情况

最坏情况下, A需要一趟最远往返, 也就是 2τ 的时间才能检测到自己发出的数据经过了碰撞

前面提到, 碰撞检测是只有在发送的时候才会检测的, 那么如果我们在最坏的 2τ 时间之前就停止了发送数据

数据一旦发生碰撞我们就无法检测到

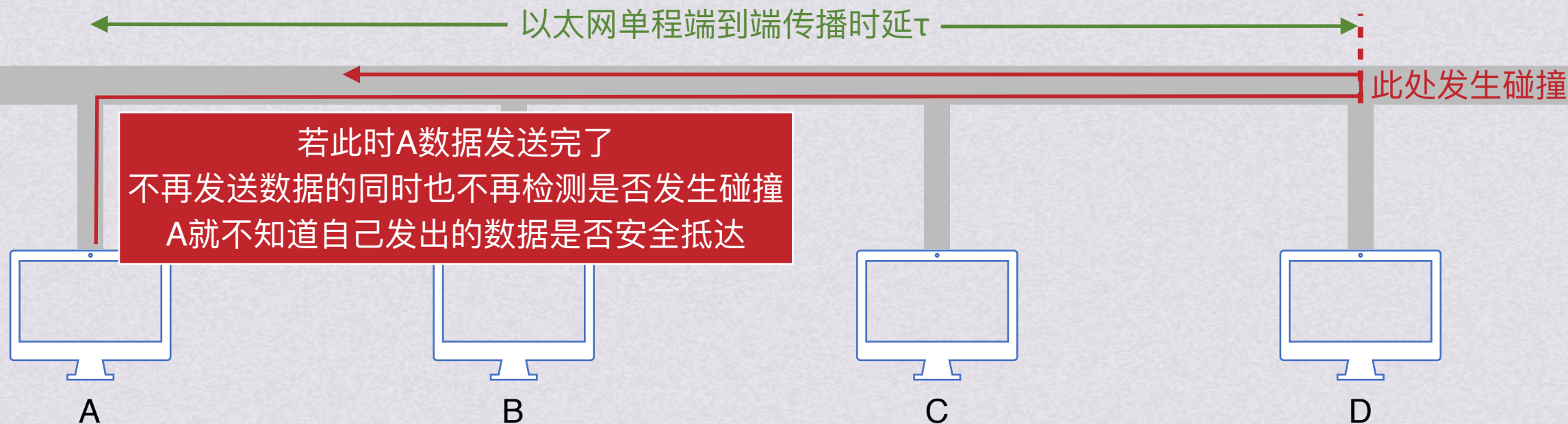


随机接入

CSMA/CD协议

争用期(碰撞窗口) τ : 以太网单程端到端传播时延

假设D在A发送的数据即将到达D的时候才发送数据



因为不发送数据就无法检测是否发生碰撞；只有经过 2τ 时间才能确认是否发生碰撞
所以引入了最短帧长的概念，确保发送一个帧的时间要 $\geq 2\tau$

最短帧长 $= 2 * \text{总线传播时延} * \text{数据传输速率} = 2\tau * \text{数据传输速率}$

以太网规定最短帧长为64B，即512bit，若数据太短则需要填充到这么长才能发送

若在争用期内检测到碰撞，会停止发送，此时发出去的数据 $< 64\text{B}$

所以凡是 $< 64\text{B}$ 的帧都是因为碰撞中止的无效帧



随机接入

CSMA/CD协议

争用期(碰撞窗口) τ : 以太网单程端到端传播时延

因为不发送数据就无法检测是否发生碰撞; 只有经过 2τ 时间才能确认是否发生碰撞
所以引入了最短帧长的概念, 确保发送一个帧的时间要 $\geq 2\tau$

最短帧长 $= 2 * \text{总线传播时延} * \text{数据传输速率} = 2\tau * \text{数据传输速率}$

以太网规定最短帧长为64B, 即512bit, 若数据太短则需要填充到这么长才能发送

若在争用期内检测到碰撞, 会停止发送, 此时发出去的数据 $< 64\text{B}$

所以凡是 $< 64\text{B}$ 的帧都是因为碰撞中止的无效帧

但是数据帧长度也不能无限长, 否则会让总线上其他主机无法发送信息

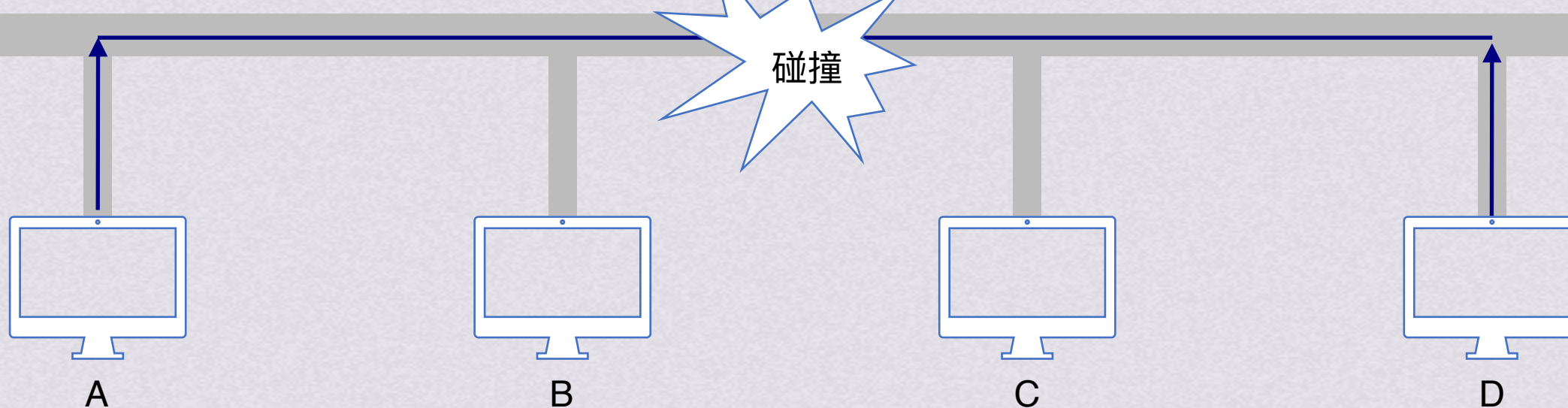
以太网V2的MAC帧(最大长度1518B)					
目的地址	源地址	类型	数据载荷	FCS	
字节 6B	6B	2B	46~1500B	4B	

插入VLAN标记后的802.1Q帧(最大长度1522B)					
目的地址	源地址	VLAN标记	类型	数据载荷	FCS
字节 6B	6B	4B	2B	46~1500B	4B



随机接入

CSMA/CD协议



在检测到碰撞时，适配器会执行**指数退避算法**来等待一段随机事件

指数退避算法：基本退避时间为争用期 2τ ，在以太网中是发送64B(512b)所需时间51.2us

从整数集合 $[0, 1, \dots, 2^k - 1]$ 中随机取一个数，记为 r 。重传应推后的时间是 r 倍的争用期($r\tau$)

整数集合里的 $k = \text{Min}[\text{重传次数}, 10]$

如果重传16次还不成功，则丢弃该帧，向高层报告



随机接入

CSMA/CD协议

在检测到碰撞时，适配器会执行**指数退避算法**来等待一段随机事件

指数退避算法：基本退避时间为争用期 2τ ，在以太网中是发送64B(512b)所需时间51.2us

从整数集合 $[0, 1, \dots, 2^k - 1]$ 中随机取一个数，记为 r 。重传应推后的时间是 r 倍的争用期($r\tau$)

整数集合里的 $k = \text{Min}[\text{重传次数}, 10]$

如果重传16次还不成功，则丢弃该帧，向高层报告

已知10BaseT以太网的争用时间片为51.2us。若网卡在发送某帧时发生了连续4次冲突，则基于二进制指数退避算法确定的再次尝试重发该帧前等待的最长时间是()

A.51.2us

B.204.8us

C.768us

D.819.2us



随机接入

CSMA/CD协议

在检测到碰撞时，适配器会执行**指数退避算法**来等待一段随机事件

指数退避算法：基本退避时间为争用期 2τ ，在以太网中是发送64B(512b)所需时间51.2us

从整数集合 $[0, 1, \dots, 2^k - 1]$ 中随机取一个数，记为 r 。重传应推后的时间是 r 倍的争用期($r\tau$)

整数集合里的 $k = \text{Min}[\text{重传次数}, 10]$

如果重传16次还不成功，则丢弃该帧，向高层报告

已知10BaseT以太网的争用时间片为51.2us。若网卡在发送某帧时发生了连续4次冲突，则基于二进制指数退避算法确定的再次尝试重发该帧前等待的最长时间是(**C.768us**)

$k = \text{Min}[\text{重传次数}, 10] = 4$

从集合 $[0, 1, \dots, 2^4 - 1] = [0, 1, \dots, 15]$ 中随机取数作为 r

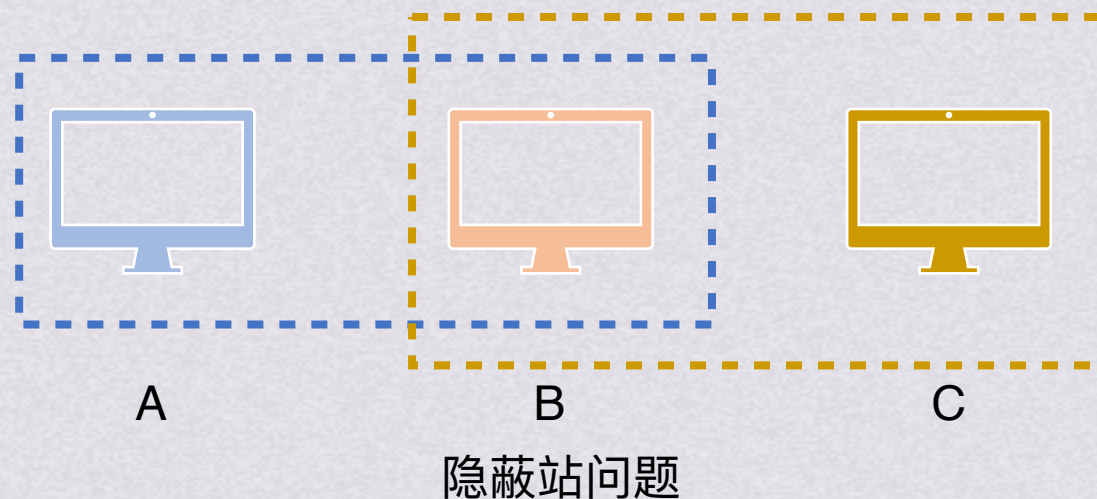
则最长的等待时间是 15τ ，以太网的 $\tau = 51.2\text{us}$ ，则 $15\tau = 15 * 51.2\text{us} = 768\text{us}$



随机接入

CSMA/CA协议

CSMA/CD主要用于有线连接的局域网，但是对于无线局域网不使用，因为无线局域网有隐蔽站问题，导致碰撞检测意义不大



这种情况下A和C都能检测到并且和C通信，B也能和AC通信，但是A不能检测到C，C也无法检测到A

A和C都给B发送帧时就发生了碰撞，并且A和C无法检测到碰撞

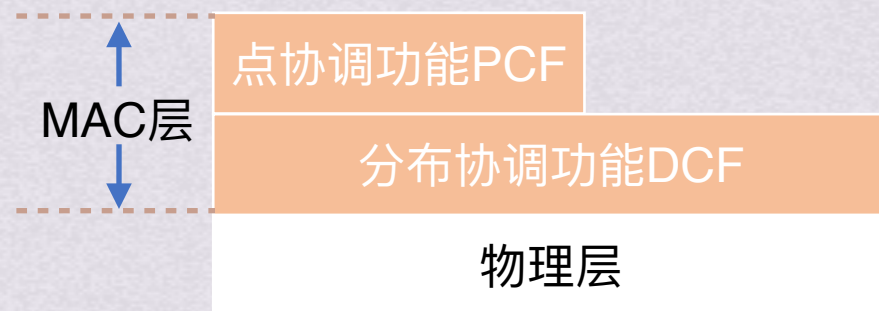


随机接入

CSMA/CA协议

CSMA/CD主要用于有线连接的局域网，但是对于无线局域网不使用，因为无线局域网有隐蔽站问题，导致碰撞检测意义不大
因此在无线局域网改为使用CSMA/CA协议，CA(Collision Avoidance)即碰撞避免，在CSMA的基础上增加了碰撞避免的功能
因为无线信道的通信质量远不如有线信道，802.11局域网使用CSMA/CA的同时，还使用停止等待协议

802.11标准通过协调功能确定移动站在什么时间能发送数据或接受数据。802.11的MAC层包括两个子层



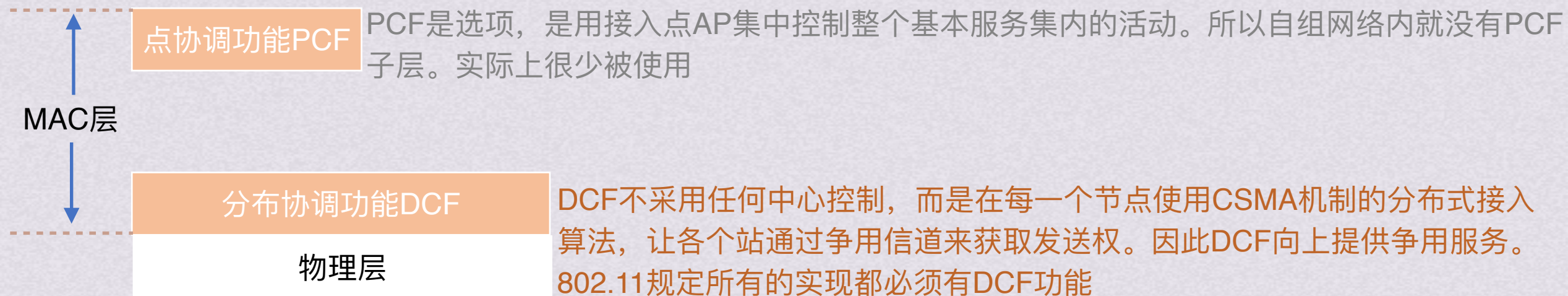


随机接入

CSMA/CA协议

CSMA/CD主要用于有线连接的局域网，但是对于无线局域网不使用，因为无线局域网有隐蔽站问题，导致碰撞检测意义不大
因此在无线局域网改为使用CSMA/CA协议，CA(Collision Avoidance)即碰撞避免，在CSMA的基础上增加了碰撞避免的功能
因为无线信道的通信质量远不如有线信道，802.11局域网使用CSMA/CA的同时，还使用停止等待协议

802.11标准通过协调功能确定移动站在什么时间能发送数据或接受数据。802.11的MAC层包括两个子层





随机接入

CSMA/CA协议

CSMA/CD主要用于有线连接的局域网，但是对于无线局域网不使用，因为无线局域网有隐蔽站问题，导致碰撞检测意义不大
因此在无线局域网改为使用CSMA/CA协议，CA(Collision Avoidance)即碰撞避免，在CSMA的基础上增加了碰撞避免的功能
因为无线信道的通信质量远不如有线信道，802.11局域网使用CSMA/CA的同时，还使用停止等待协议

802.11规定，所有站必须在检测到信道空闲一段时间后才能发送一帧。这一段空闲时间称为帧间间隔IFS

IFS的长度取决于要发送帧的类型，高优先级的帧等待时间更短
低优先级的帧等待时间更长，因此高优先级的帧能够优先的被发送到信道上

IFS分为两种

SIFS(short IFS)短帧间间隔，长度为28us，是最短的帧间间隔，用来分隔开属于一次对话的各帧。这段时间内，一个站应当能够从发送方式切换到接收方式。使用SIFS的帧类型有：ACK帧、CTS帧、由过长的MAC帧分片后的数据帧、以及所有回答AP探测的帧和在PCF方式中接入点AP发出的任何帧

DIFS分布协调功能帧间间隔，它比SIFS的帧间间隔要长的多，长度为128us，在DCF方式中用来发送数据帧和管理帧



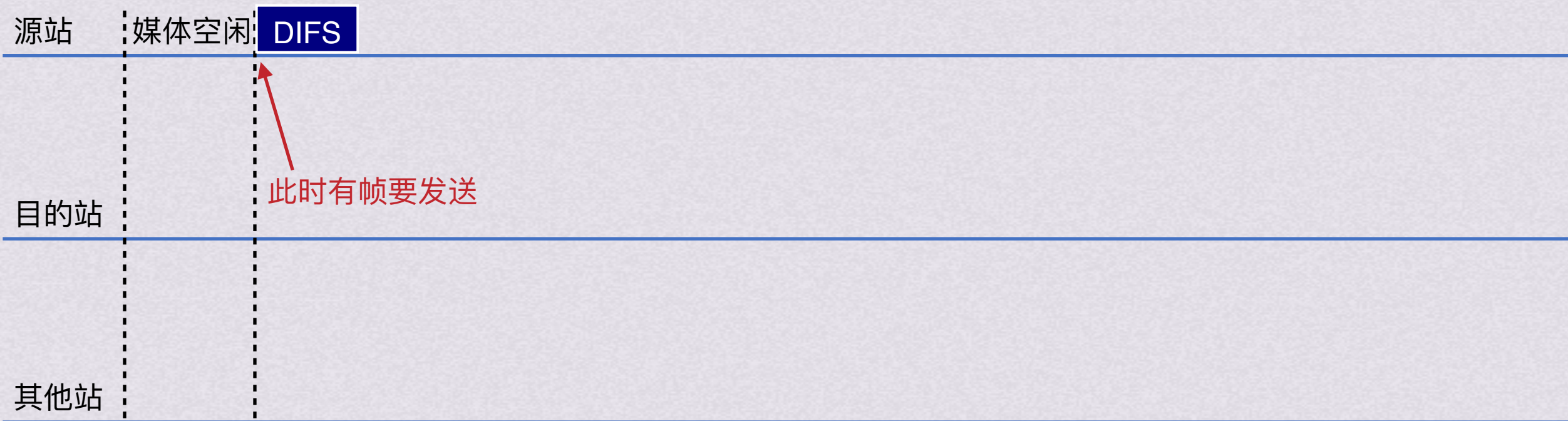
随机接入

CSMA/CA协议

IFS分为两种

SIFS(short IFS)短帧间间隔，长度为28us，是最短的帧间间隔，用来分隔开属于一次对话的各帧。这段时间内，一个站应当能够从发送方式切换到接收方式。使用SIFS的帧类型有：ACK帧、CTS帧、由过长的MAC帧分片后的数据帧、以及所有回答AP探测的帧和在PCF方式中接入点AP发出的任何帧

DIFS分布协调功能帧间间隔，它比SIFS的帧间间隔要长的多，长度为128us，在DCF方式中用来发送数据帧和管理帧





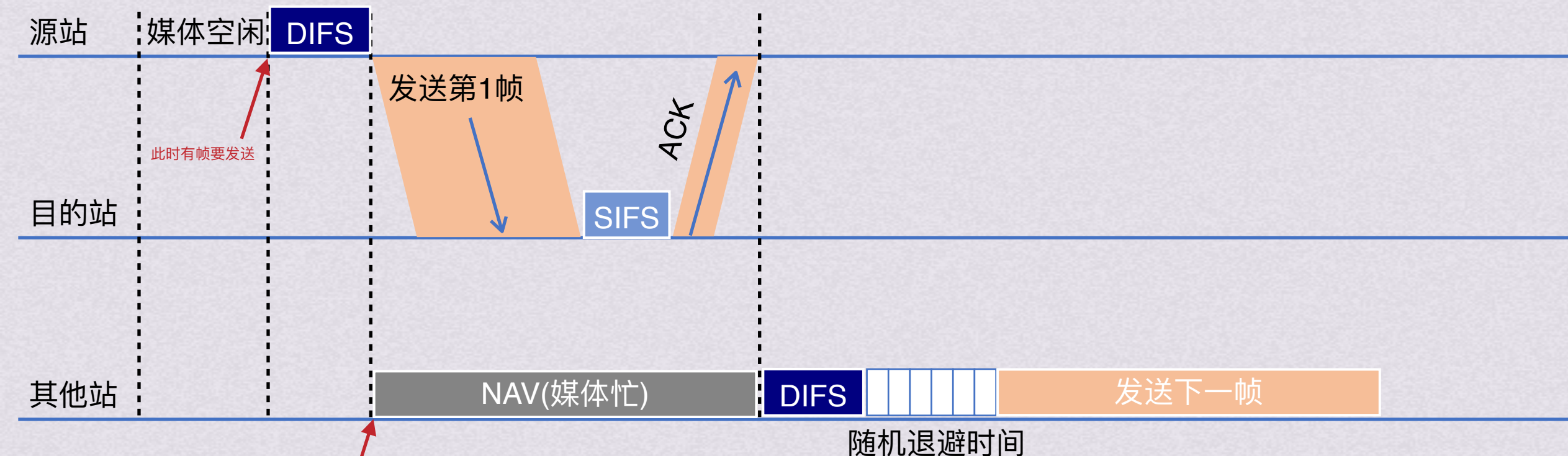
随机接入

CSMA/CA协议

IFS分为两种

SIFS(short IFS)短帧间间隔, 长度为28us, 是最短的帧间间隔, 用来分隔开属于一次对话的各帧。这段时间内, 一个站应当能够从发送方式切换到接收方式。使用SIFS的帧类型有: ACK帧、CTS帧、由过长的MAC帧分片后的数据帧、以及所有回答AP探测的帧和在PCF方式中接入点AP发出的任何帧

DIFS分布协调功能帧间间隔, 它比SIFS的帧间间隔要长的多, 长度为128us, 在DCF方式中用来发送数据帧和管理帧



此时有帧要发送, 因为检测到媒体忙推迟接入



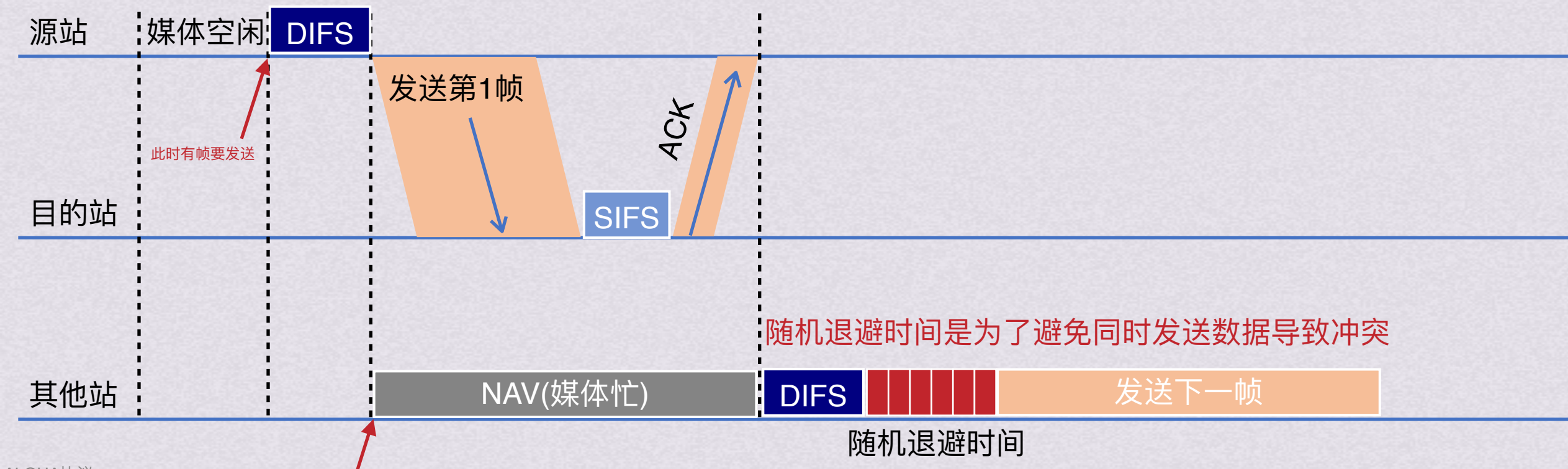
随机接入

CSMA/CA协议

IFS分为两种

SIFS(short IFS)短帧间间隔, 长度为28us, 是最短的帧间间隔, 用来分隔开属于一次对话的各帧。这段时间内, 一个站应当能够从发送方式切换到接收方式。使用SIFS的帧类型有: ACK帧、CTS帧、由过长的MAC帧分片后的数据帧、以及所有回答AP探测的帧和在PCF方式中接入点AP发出的任何帧

DIFS分布协调功能帧间间隔, 它比SIFS的帧间间隔要长的多, 长度为128us, 在DCF方式中用来发送数据帧和管理帧





随机接入

CSMA/CA协议

IFS分为两种

SIFS(short IFS)短帧间间隔, 长度为28us, 是最短的帧间间隔, 用来分隔开属于一次对话的各帧。这段时间内, 一个站应当能够从发送方式切换到接收方式。使用SIFS的帧类型有: ACK帧、CTS帧、由过长的MAC帧分片后的数据帧、以及所有回答AP探测的帧和在PCF方式中接入点AP发出的任何帧

DIFS分布协调功能帧间间隔, 它比SIFS的帧间间隔要长的多, 长度为128us, 在DCF方式中用来发送数据帧和管理帧

随机退避时间是为了避免同时发送数据导致冲突

其他站



随机退避时间

当站点检测到信道空闲, 并且所发送数据帧不是发完上一个数据帧以后立即连续发送的数据帧, 就不使用退避算法

以下情况必须使用退避算法:

在发送数据帧之前检测到信道处于忙状态时

在每一次重传一个数据帧时

在每一次成功发送后要连续发送下一个帧时(这样可以避免一个站点长期占用信道)



随机接入

CSMA/CA协议

当站点检测到信道空闲，并且所发送数据帧不是发完上一个数据帧以后立即连续发送的数据帧，就不使用退避算法

以下情况必须使用退避算法：在发送数据帧之前检测到信道处于忙状态时

在每一次重传一个数据帧时

在每一次成功发送后要连续发送下一个帧时(这样可以避免一个站点长期占用信道)

CSMA/CA协议的退避算法：

执行退避算法时，站点为退避计时器设置一个随机的退避时间

当退避计时器的时间减小到0时，就开始发送数据

当退避计时器的时间还未减小到0时，而信道又变成忙状态，这时冻结退避计时器，重新等待信道变成空闲再经过DIFS后，再启动退避计时器

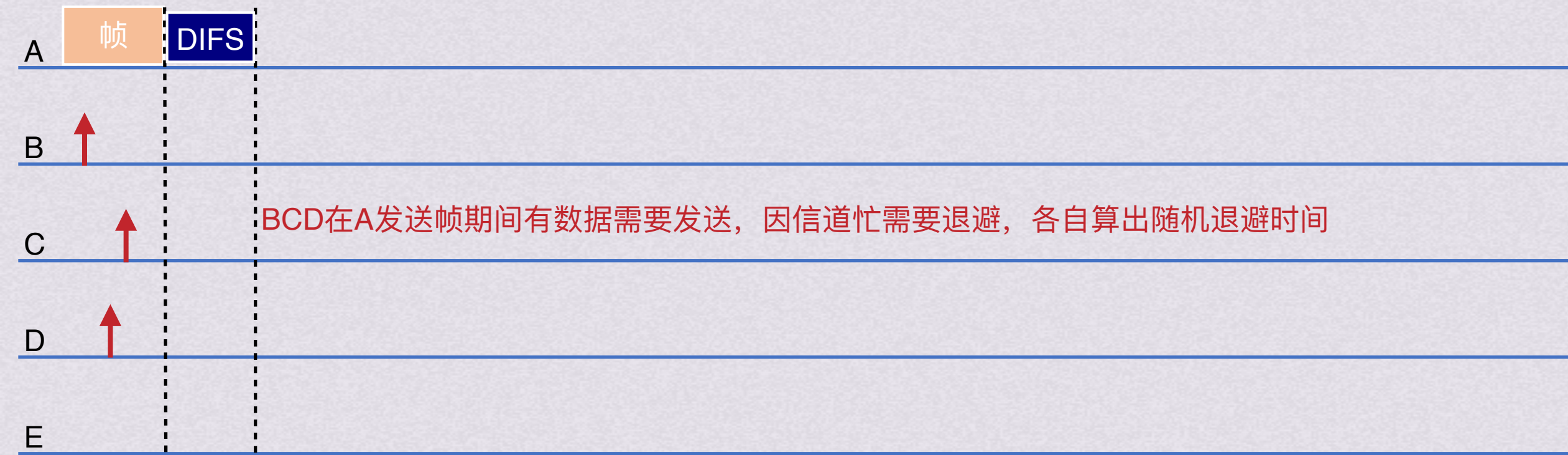
在第*i*次退避时，在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个，随机乘以基本退避时间(一个时隙长度)就得到随机退避时间



随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时，站点为退避计时器设置一个随机的退避时间
 当退避计时器的时间减小到0时，就开始发送数据
 当退避计时器的时间还未减小到0时，而信道又变成忙状态，这时冻结退避计时器，
 重新等待信道变成空闲再经过DIFS后，再启动退避计时器
 在第*i*次退避时，在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个，随机乘以基本退避时间(一个时隙长度)就得到随机退避时间

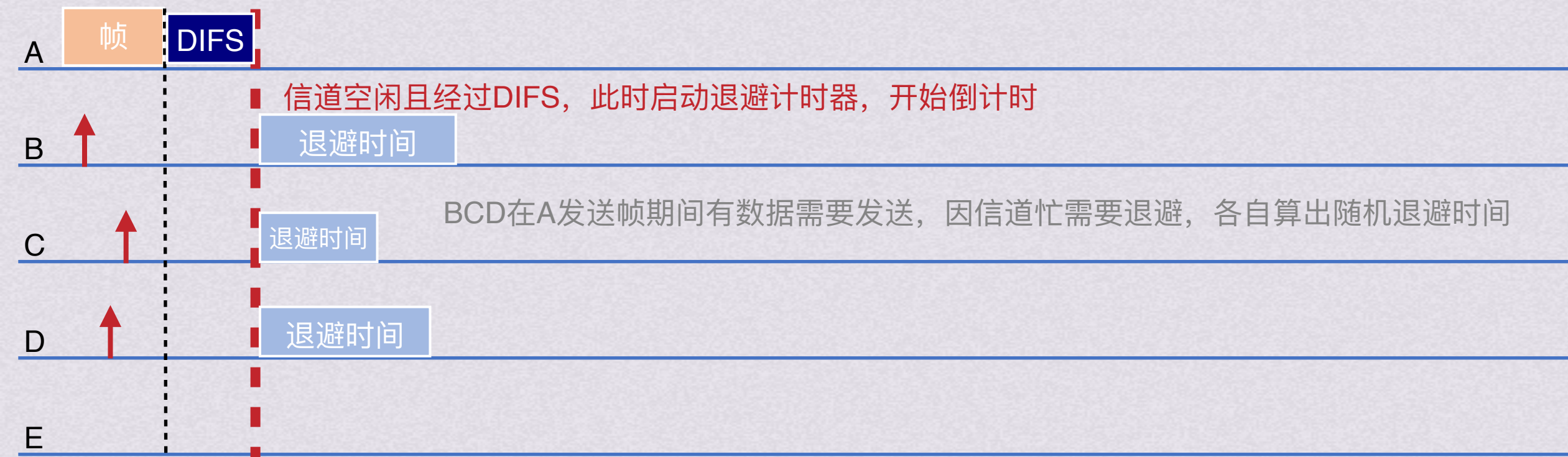




随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时，站点为退避计时器设置一个随机的退避时间
 当退避计时器的时间减小到0时，就开始发送数据
 当退避计时器的时间还未减小到0时，而信道又变成忙状态，这时冻结退避计时器，重新等待信道变成空闲再经过DIFS后，再启动退避计时器
 在第*i*次退避时，在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个，随机乘以基本退避时间(一个时隙长度)就得到随机退避时间





随机接入

CSMA/CA协议

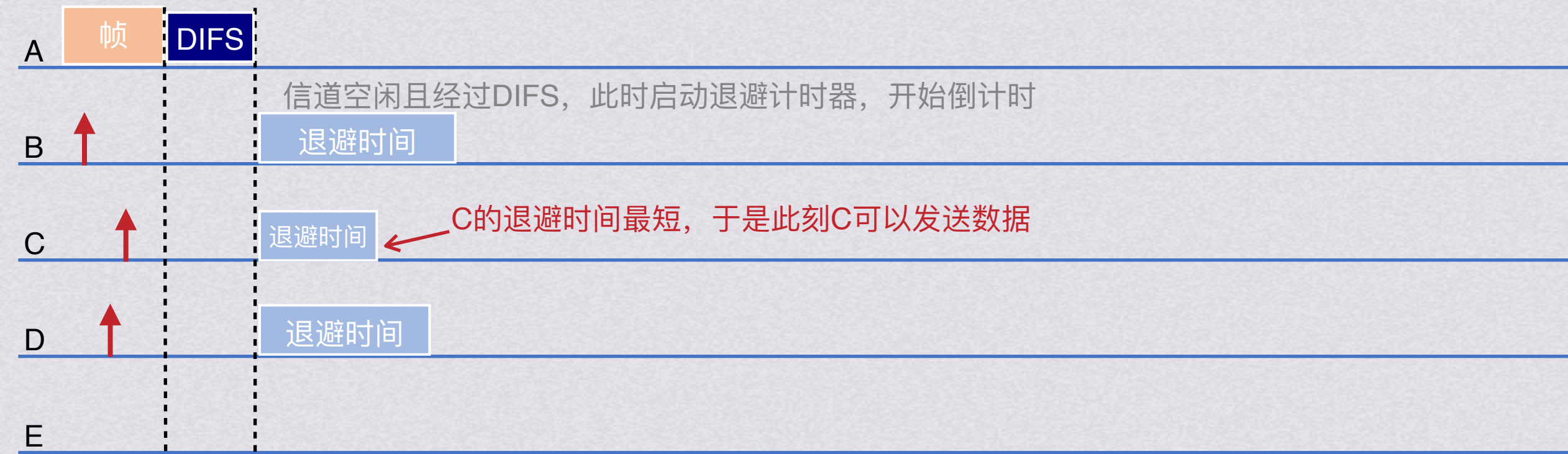
CSMA/CA协议的退避算法: 执行退避算法时, 站点为退避计时器设置一个随机的退避时间

当退避计时器的时间减小到0时, 就开始发送数据

当退避计时器的时间还未减小到0时, 而信道又变成忙状态, 这时冻结退避计时器,

重新等待信道变成空闲再经过DIFS后, 再启动退避计时器

在第*i*次退避时, 在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个, 随机乘以基本退避时间(一个时隙长度)就得到随机退避时间

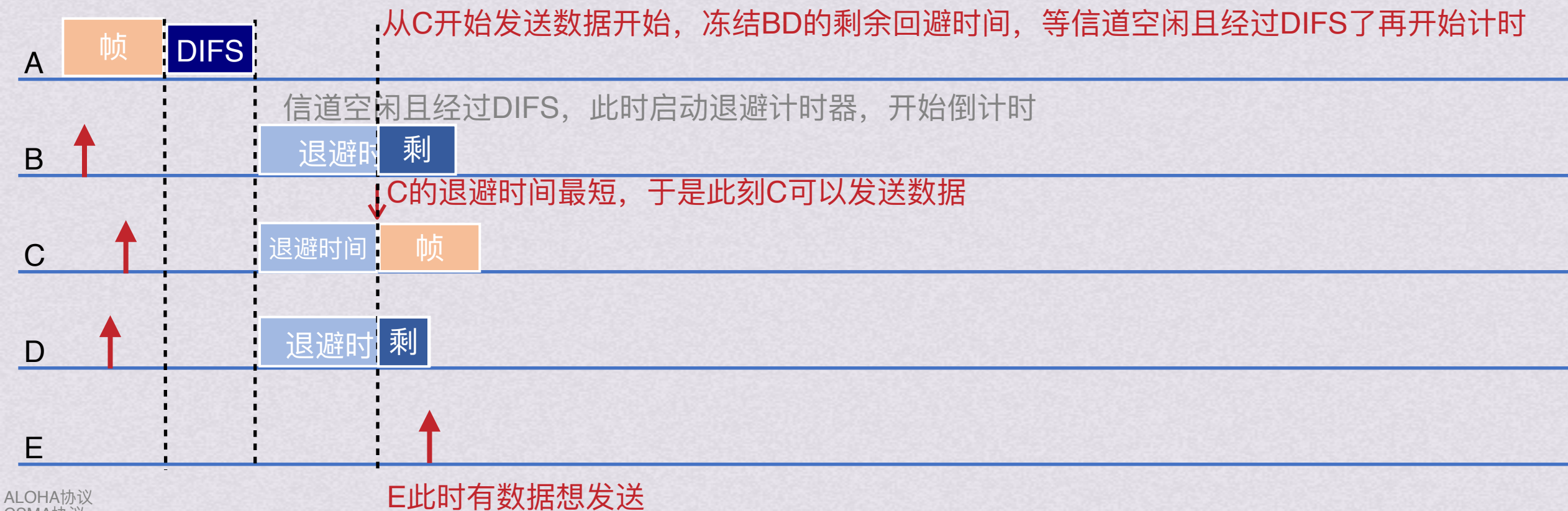




随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时，站点为退避计时器设置一个随机的退避时间
 当退避计时器的时间减小到0时，就开始发送数据
 当退避计时器的时间还未减小到0时，而信道又变成忙状态，这时冻结退避计时器，重新等待信道变成空闲再经过DIFS后，再启动退避计时器
 在第*i*次退避时，在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个，随机乘以基本退避时间(一个时隙长度)就得到随机退避时间





随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时, 站点为退避计时器设置一个随机的退避时间

当退避计时器的时间减小到0时, 就开始发送数据

当退避计时器的时间还未减小到0时, 而信道又变成忙状态, 这时冻结退避计时器,

重新等待信道变成空闲再经过DIFS后, 再启动退避计时器

在第*i*次退避时, 在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个, 随机乘以基本退避时间(一个时隙长度)就得到随机退避时间

从C开始发送数据开始, 冻结BD的剩余回避时间, 等信道空闲且经过DIFS了再开始计时



E此时有数据想发送



随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时, 站点为退避计时器设置一个随机的退避时间

当退避计时器的时间减小到0时, 就开始发送数据

当退避计时器的时间还未减小到0时, 而信道又变成忙状态, 这时冻结退避计时器,

重新等待信道变成空闲再经过DIFS后, 再启动退避计时器

在第*i*次退避时, 在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个, 随机乘以基本退避时间(一个时隙长度)就得到随机退避时间

从C开始发送数据开始, 冻结BD的剩余回避时间, 等信道空闲且经过DIFS了再开始计时





随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时, 站点为退避计时器设置一个随机的退避时间

当退避计时器的时间减小到0时, 就开始发送数据

当退避计时器的时间还未减小到0时, 而信道又变成忙状态, 这时冻结退避计时器,

重新等待信道变成空闲再经过DIFS后, 再启动退避计时器

在第*i*次退避时, 在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个, 随机乘以基本退避时间(一个时隙长度)就得到随机退避时间

从C开始发送数据开始, 冻结BD的剩余回避时间, 等信道空闲且经过DIFS了再开始计时





随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时, 站点为退避计时器设置一个随机的退避时间

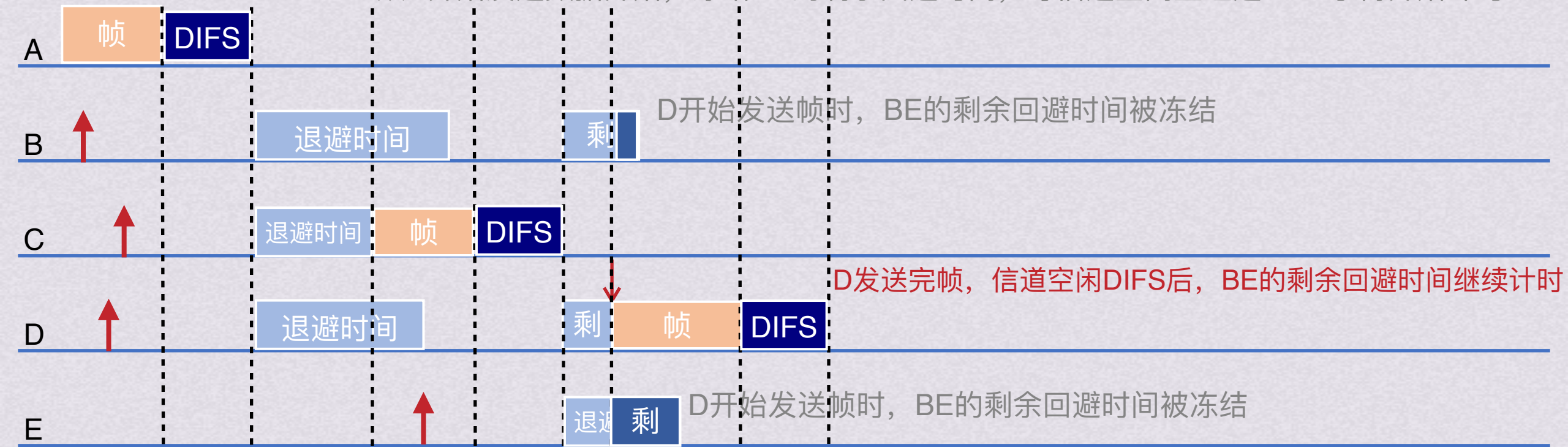
当退避计时器的时间减小到0时, 就开始发送数据

当退避计时器的时间还未减小到0时, 而信道又变成忙状态, 这时冻结退避计时器,

重新等待信道变成空闲再经过DIFS后, 再启动退避计时器

在第*i*次退避时, 在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个, 随机乘以基本退避时间(一个时隙长度)就得到随机退避时间

从C开始发送数据开始, 冻结BD的剩余回避时间, 等信道空闲且经过DIFS了再开始计时





随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时, 站点为退避计时器设置一个随机的退避时间

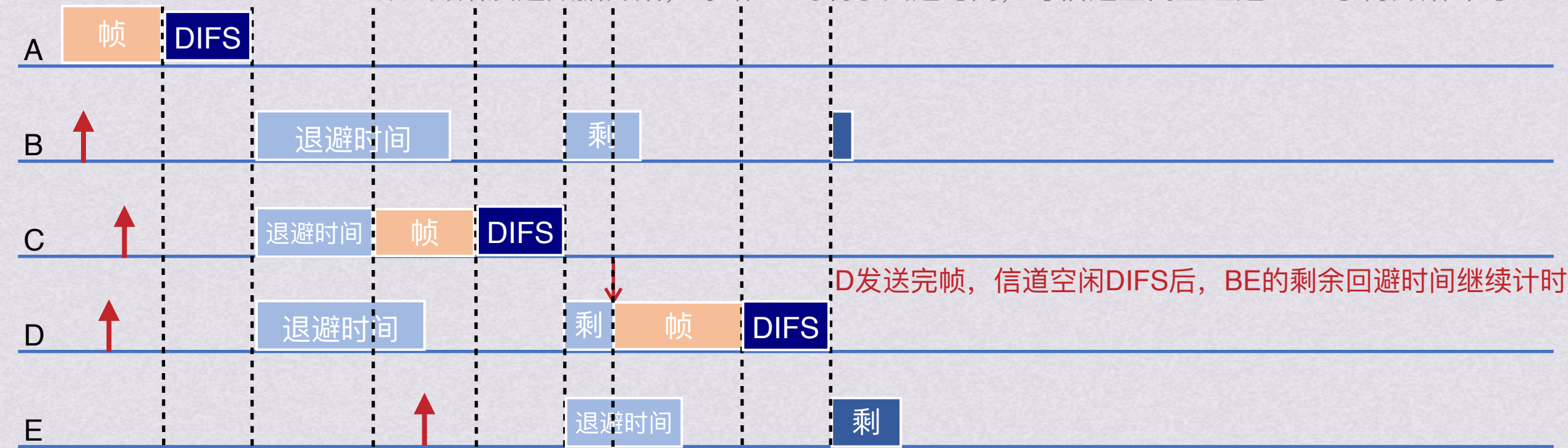
当退避计时器的时间减小到0时, 就开始发送数据

当退避计时器的时间还未减小到0时, 而信道又变成忙状态, 这时冻结退避计时器,

重新等待信道变成空闲再经过DIFS后, 再启动退避计时器

在第*i*次退避时, 在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个, 随机乘以基本退避时间(一个时隙长度)就得到随机退避时间

从C开始发送数据开始, 冻结BD的剩余回避时间, 等信道空闲且经过DIFS了再开始计时



E此时有数据想发送



随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时, 站点为退避计时器设置一个随机的退避时间

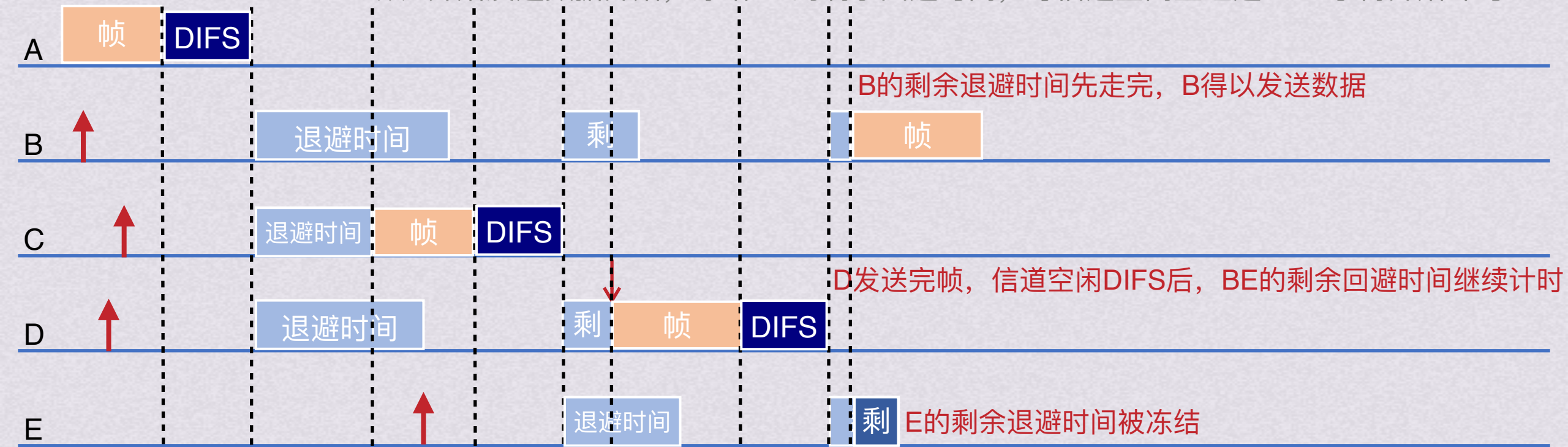
当退避计时器的时间减小到0时, 就开始发送数据

当退避计时器的时间还未减小到0时, 而信道又变成忙状态, 这时冻结退避计时器,

重新等待信道变成空闲再经过DIFS后, 再启动退避计时器

在第*i*次退避时, 在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个, 随机乘以基本退避时间(一个时隙长度)就得到随机退避时间

从C开始发送数据开始, 冻结BD的剩余回避时间, 等信道空闲且经过DIFS了再开始计时



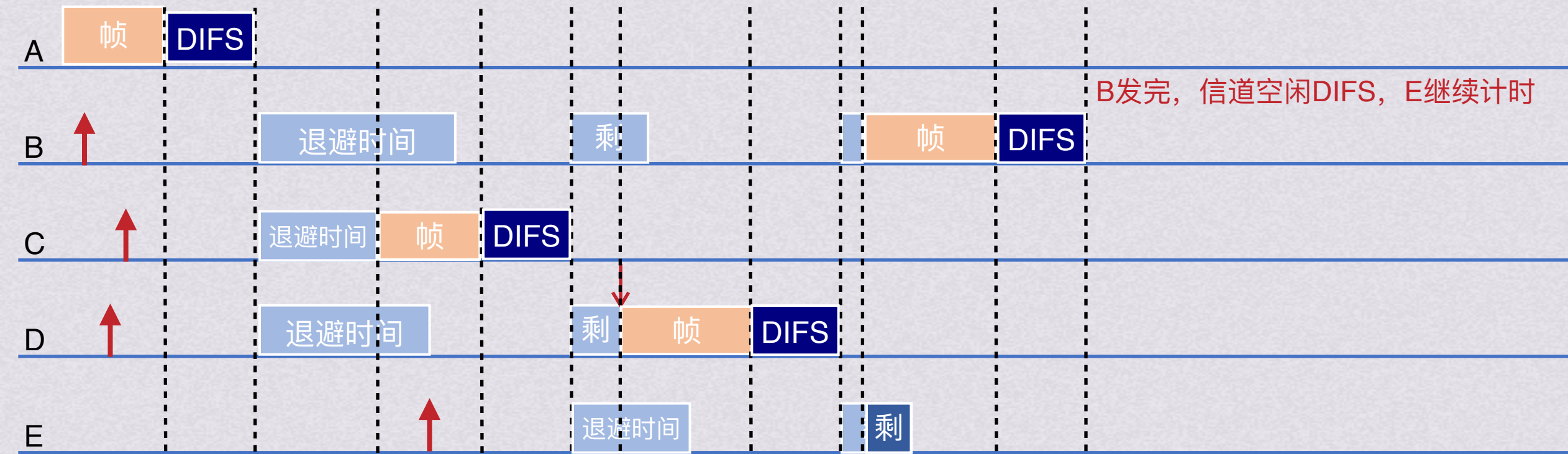
E此时有数据想发送



随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时，站点为退避计时器设置一个随机的退避时间
 当退避计时器的时间减小到0时，就开始发送数据
 当退避计时器的时间还未减小到0时，而信道又变成忙状态，这时冻结退避计时器，
 重新等待信道变成空闲再经过DIFS后，再启动退避计时器
 在第*i*次退避时，在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个，随机乘以基本退避时间(一个时隙长度)就得到随机退避时间

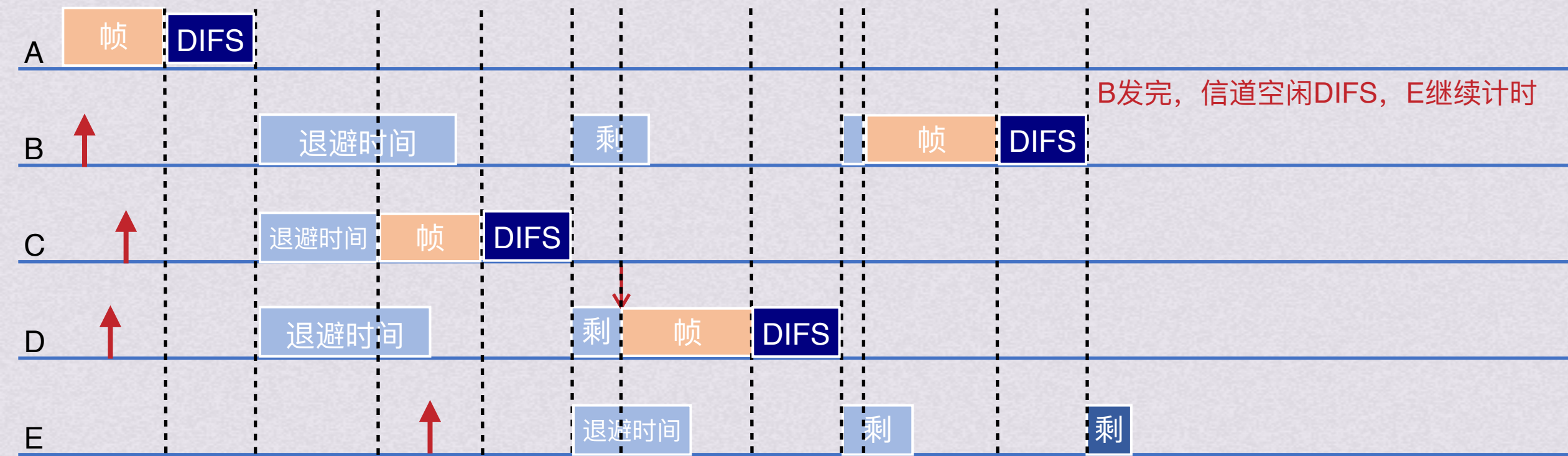




随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时，站点为退避计时器设置一个随机的退避时间
 当退避计时器的时间减小到0时，就开始发送数据
 当退避计时器的时间还未减小到0时，而信道又变成忙状态，这时冻结退避计时器，
 重新等待信道变成空闲再经过DIFS后，再启动退避计时器
 在第*i*次退避时，在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个，随机乘以基本退避时间(一个时隙长度)就得到随机退避时间



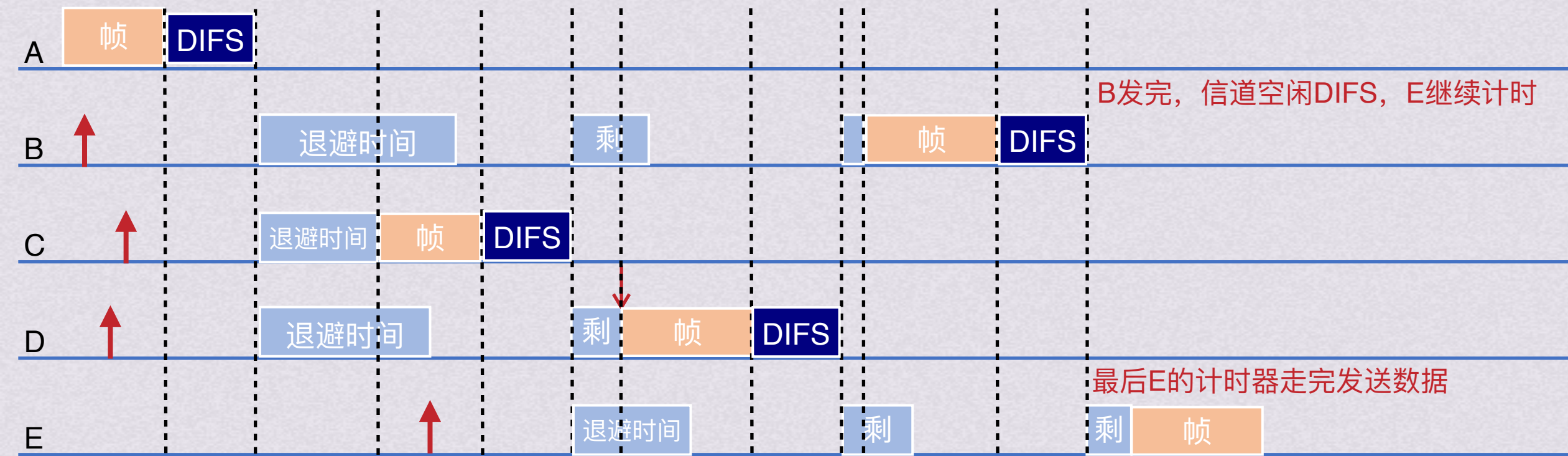
B发完，信道空闲DIFS，E继续计时



随机接入

CSMA/CA协议

CSMA/CA协议的退避算法: 执行退避算法时, 站点为退避计时器设置一个随机的退避时间
 当退避计时器的时间减小到0时, 就开始发送数据
 当退避计时器的时间还未减小到0时, 而信道又变成忙状态, 这时冻结退避计时器,
 重新等待信道变成空闲再经过DIFS后, 再启动退避计时器
 在第*i*次退避时, 在时隙 $\{0, 1, \dots, 2^{2+i} - 1\}$ 中随机选择一个, 随机乘以基本退避时间(一个时隙长度)就得到随机退避时间





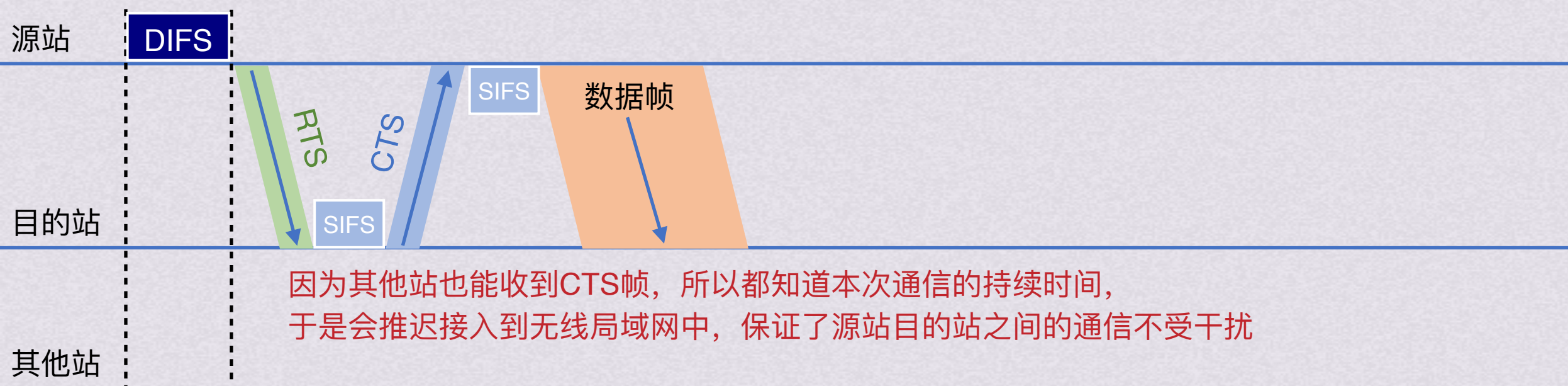
随机接入

CSMA/CA协议

为了更好地解决隐蔽战带来的碰撞问题，802.11允许要发送数据的站对信道进行预约

步骤：

- 1.A向B发送数据帧之前，先发送一个短的控制帧，称为请求发送RTS(Request To Send)，包括源地址、目的地址和本次通信需要持续时间
- 2.B如果正确收到了RTS帧，且媒体空闲，就发送一个CTS(Clear To Send)允许发送帧，也包括这次通信所需持续时间
- 3.A收到CTS后，等待一个SIFS，然后开始发送数据
- 4.B正确收到数据帧后，等待一个SIFS，然后发送确认帧ACK





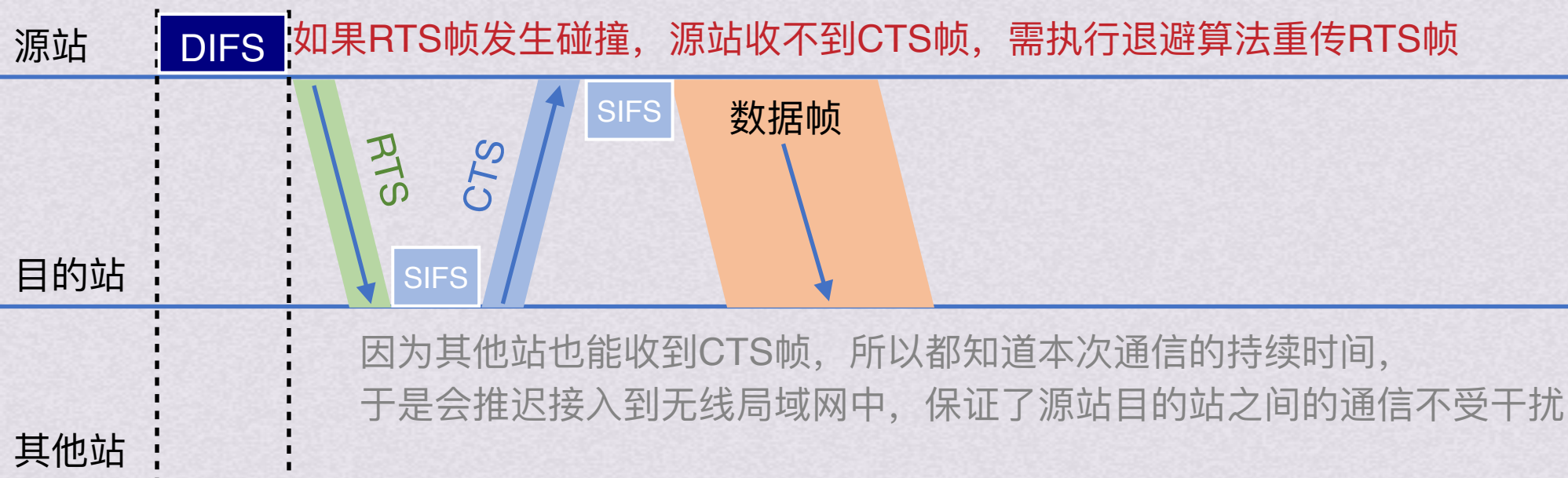
随机接入

CSMA/CA协议

为了更好地解决隐蔽战带来的碰撞问题，802.11允许要发送数据的站对信道进行预约

步骤：

- 1.A向B发送数据帧之前，先发送一个短的控制帧，称为请求发送RTS(Request To Send)，包括源地址、目的地址和本次通信需要持续时间
- 2.B如果正确收到了RTS帧，且媒体空闲，就发送一个CTS(Clear To Send)允许发送帧，也包括这次通信所需持续时间
- 3.A收到CTS后，等待一个SIFS，然后开始发送数据
- 4.B正确收到数据帧后，等待一个SIFS，然后发送确认帧ACK





随机接入

CSMA/CA协议

为了更好解决隐蔽站带来的碰撞问题，802.11允许要发送数据的站对信道进行预约

步骤：

- 1.A向B发送数据帧之前，先发送一个短的控制帧，称为请求发送RTS(Request To Send)，包括源地址、目的地址和本次通信需要持续时间
- 2.B如果正确收到了RTS帧，且媒体空闲，就发送一个CTS(Clear To Send)允许发送帧，也包括这次通信所需持续时间
- 3.A收到CTS后，等待一个SIFS，然后开始发送数据
- 4.B正确收到数据帧后，等待一个SIFS，然后发送确认帧ACK

除了RTS和CTS,数据帧也能携带通信需要时间，这称为802.11的虚拟载波监听机制
这种机制能减少隐蔽站带来的碰撞问题





动态分配信道

受控接入

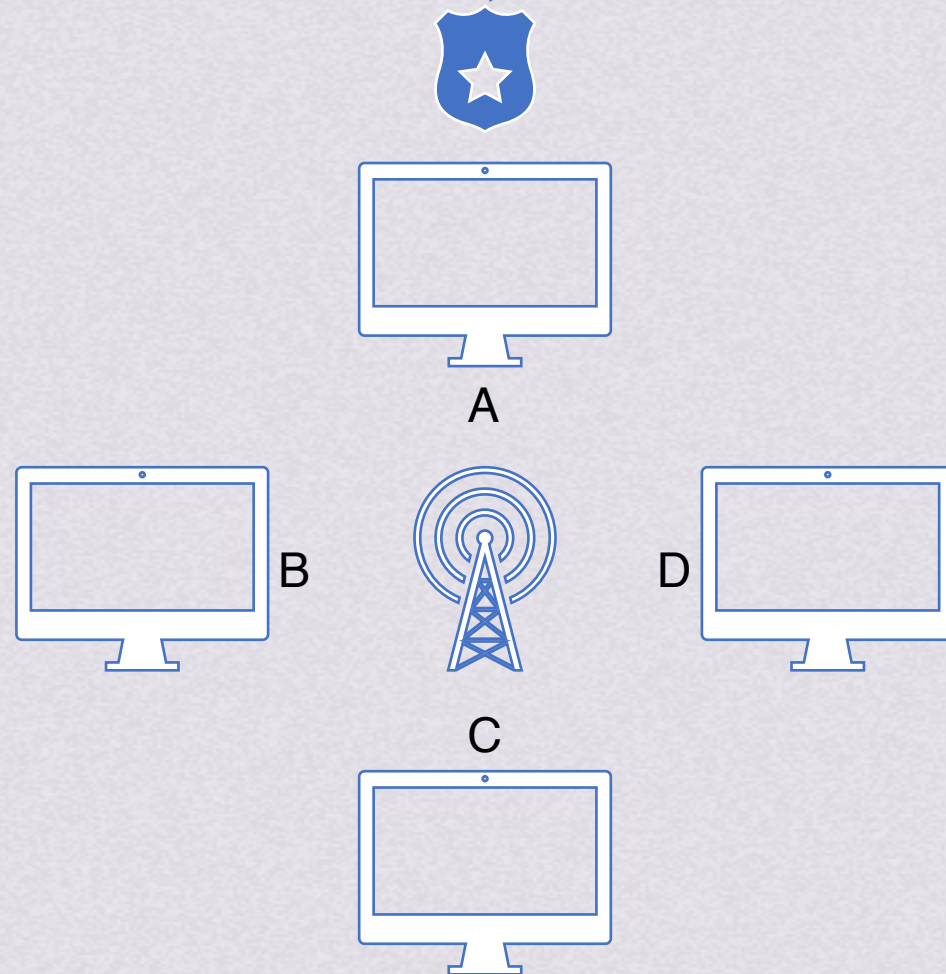


受控接入

令牌传递协议

轮训访问中，用户不能随机发送信息，必须经过集中控制的监控站，循环方式轮询每个节点。典型的轮训访问控制协议是令牌传递协议

令牌传递协议中，一个令牌沿着环形总线在各站之间依次传递。令牌是一个特殊的控制帧，本身不含信息，只控制信道使用，确保同一时刻只有一个站独占信道。站点发完一帧后，就释放令牌让其他站使用



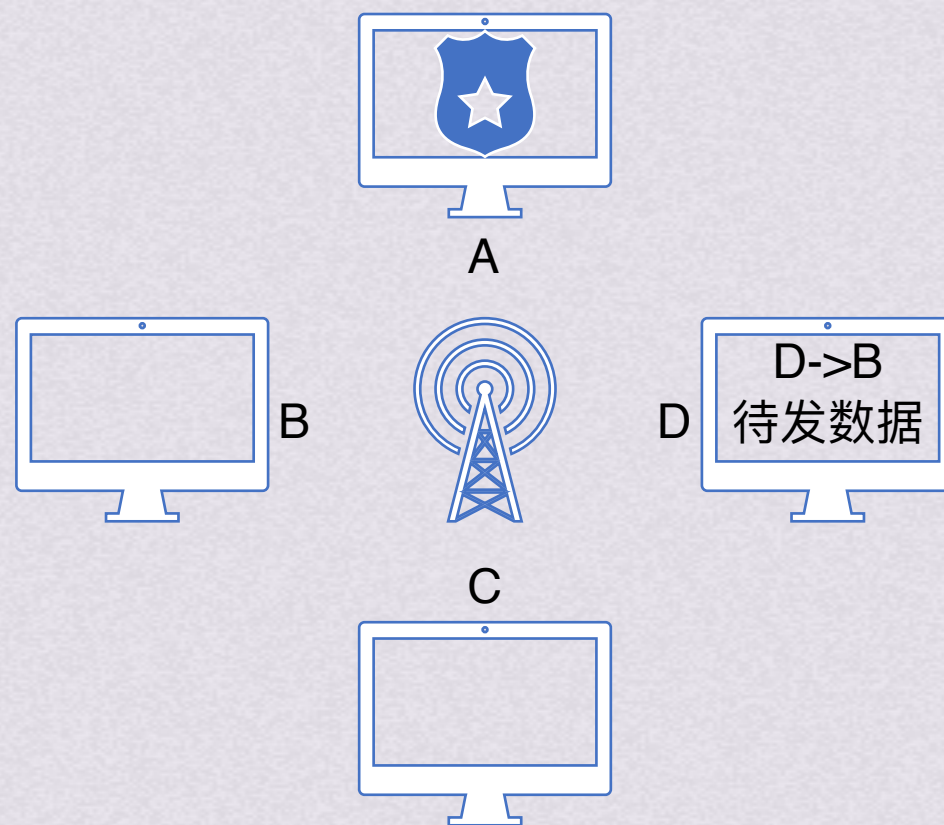


受控接入

令牌传递协议

轮训访问中，用户不能随机发送信息，必须经过集中控制的监控站，循环方式轮询每个节点。典型的轮训访问控制协议是令牌传递协议

令牌传递协议中，一个令牌沿着环形总线在各站之间依次传递。令牌是一个特殊的控制帧，本身不含信息，只控制信道使用，确保同一时刻只有一个站独占信道。站点发完一帧后，就释放令牌让其他站使用



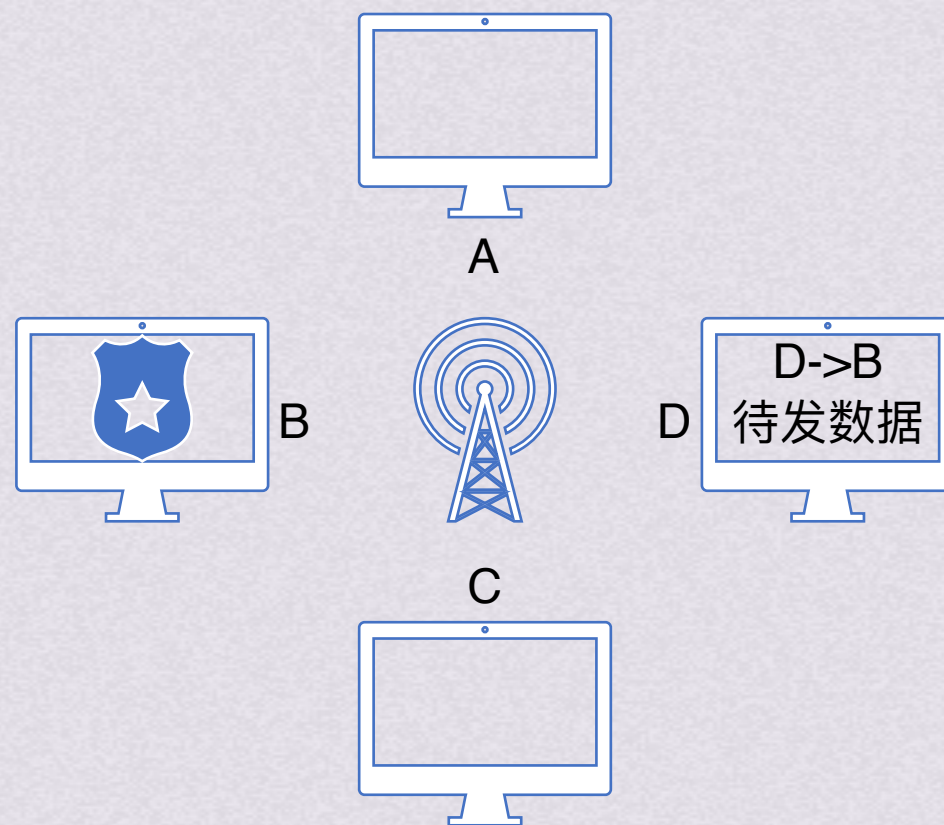


受控接入

令牌传递协议

轮训访问中，用户不能随机发送信息，必须经过集中控制的监控站，循环方式轮询每个节点。典型的轮训访问控制协议是令牌传递协议

令牌传递协议中，一个令牌沿着环形总线在各站之间依次传递。令牌是一个特殊的控制帧，本身不含信息，只控制信道使用，确保同一时刻只有一个站独占信道。站点发完一帧后，就释放令牌让其他站使用



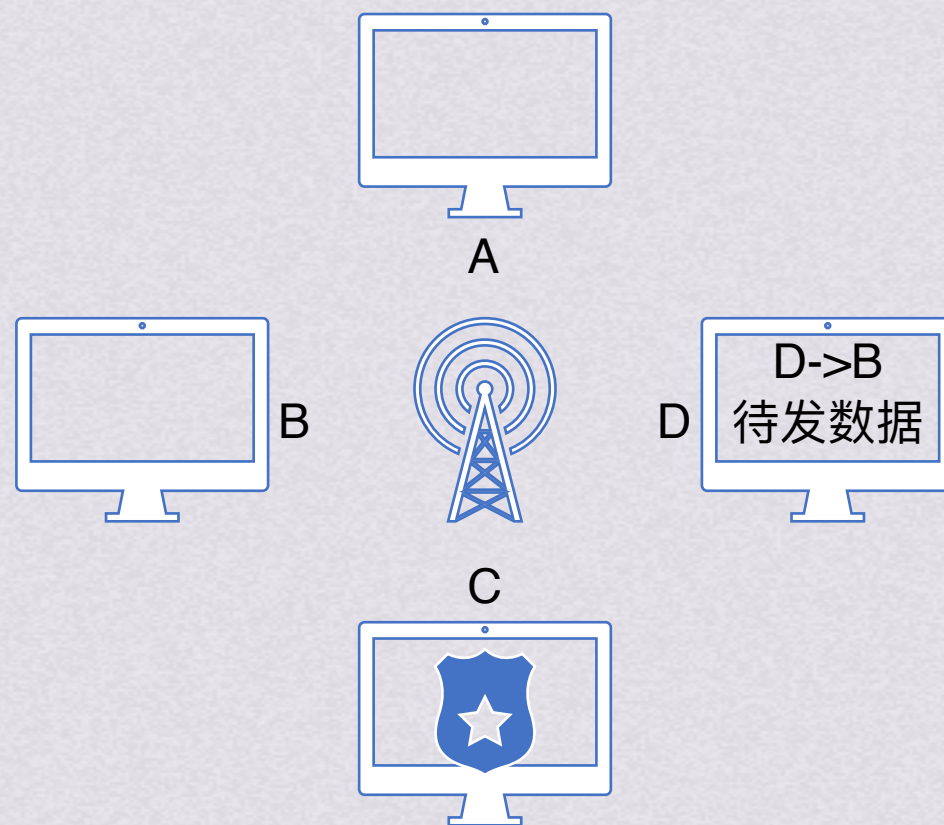


受控接入

令牌传递协议

轮训访问中，用户不能随机发送信息，必须经过集中控制的监控站，循环方式轮询每个节点。典型的轮训访问控制协议是令牌传递协议

令牌传递协议中，一个令牌沿着环形总线在各站之间依次传递。令牌是一个特殊的控制帧，本身不含信息，只控制信道使用，确保同一时刻只有一个站独占信道。站点发完一帧后，就释放令牌让其他站使用



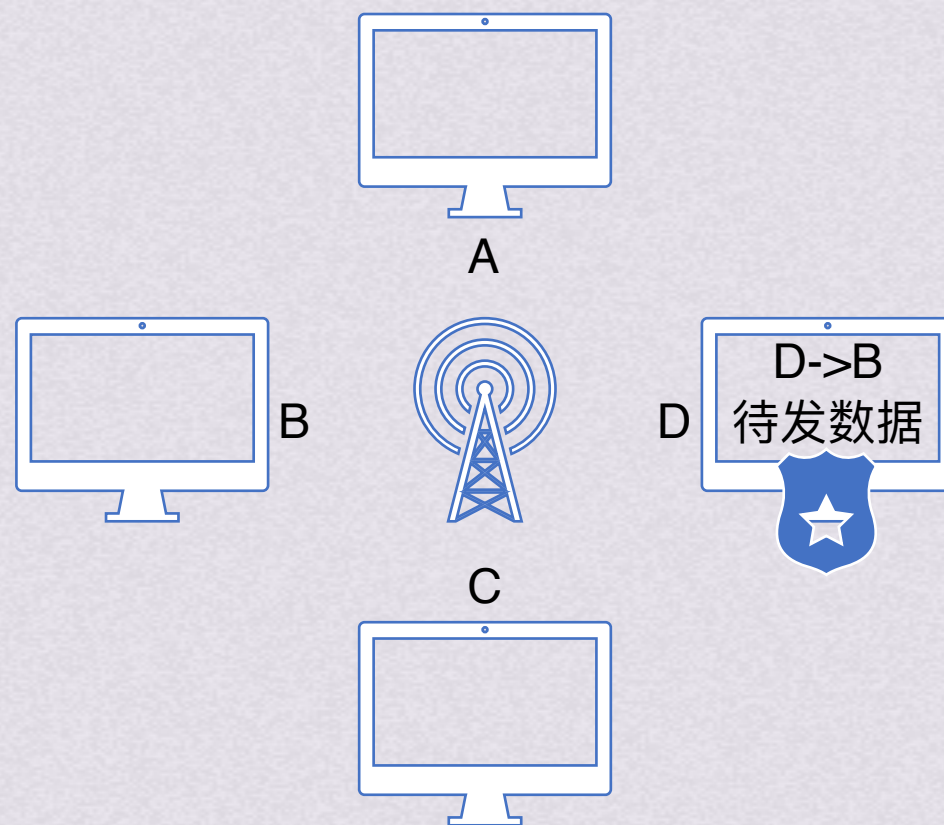


受控接入

令牌传递协议

轮训访问中，用户不能随机发送信息，必须经过集中控制的监控站，循环方式轮询每个节点。典型的轮训访问控制协议是令牌传递协议

令牌传递协议中，一个令牌沿着环形总线在各站之间依次传递。令牌是一个特殊的控制帧，本身不含信息，只控制信道使用，确保同一时刻只有一个站独占信道。站点发完一帧后，就释放令牌让其他站使用



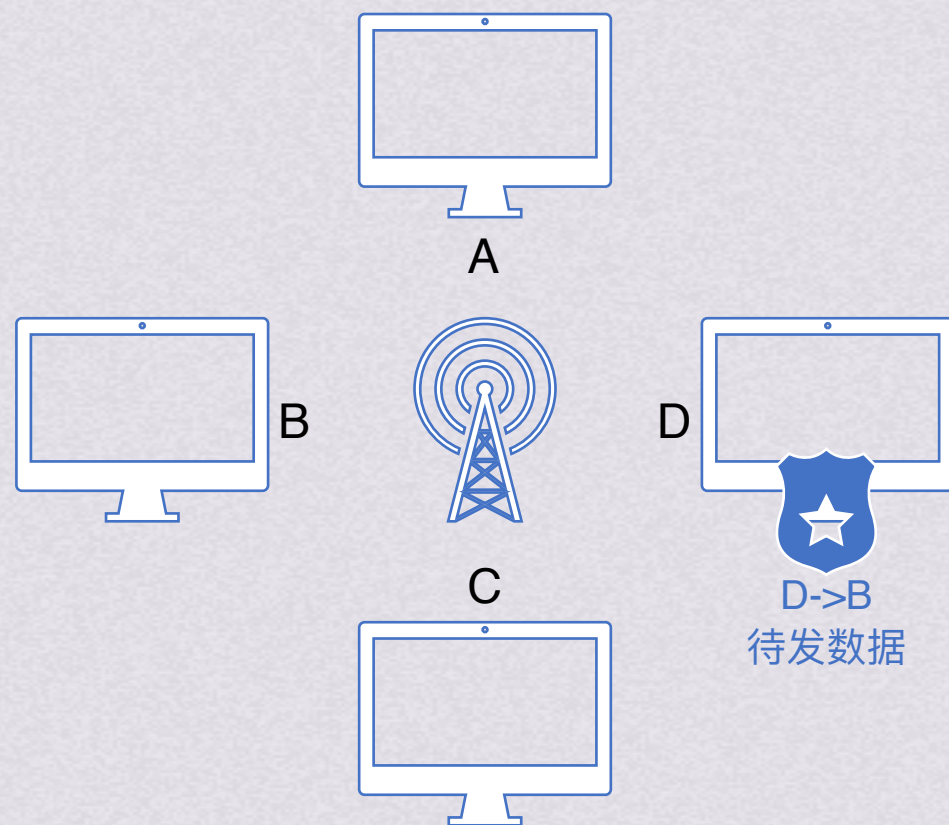


受控接入

令牌传递协议

轮训访问中，用户不能随机发送信息，必须经过集中控制的监控站，循环方式轮询每个节点。典型的轮训访问控制协议是令牌传递协议

令牌传递协议中，一个令牌沿着环形总线在各站之间依次传递。令牌是一个特殊的控制帧，本身不含信息，只控制信道使用，确保同一时刻只有一个站独占信道。站点发完一帧后，就释放令牌让其他站使用



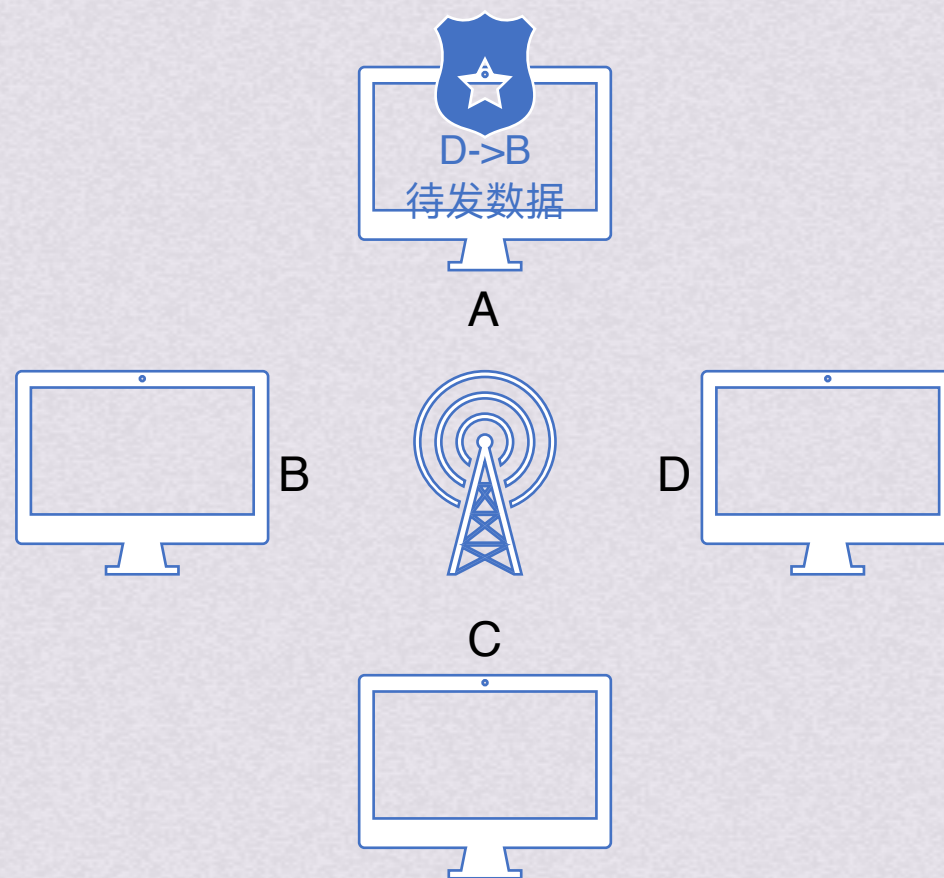


受控接入

令牌传递协议

轮训访问中，用户不能随机发送信息，必须经过集中控制的监控站，循环方式轮询每个节点。典型的轮训访问控制协议是令牌传递协议

令牌传递协议中，一个令牌沿着环形总线在各站之间依次传递。令牌是一个特殊的控制帧，本身不含信息，只控制信道使用，确保同一时刻只有一个站独占信道。站点发完一帧后，就释放令牌让其他站使用



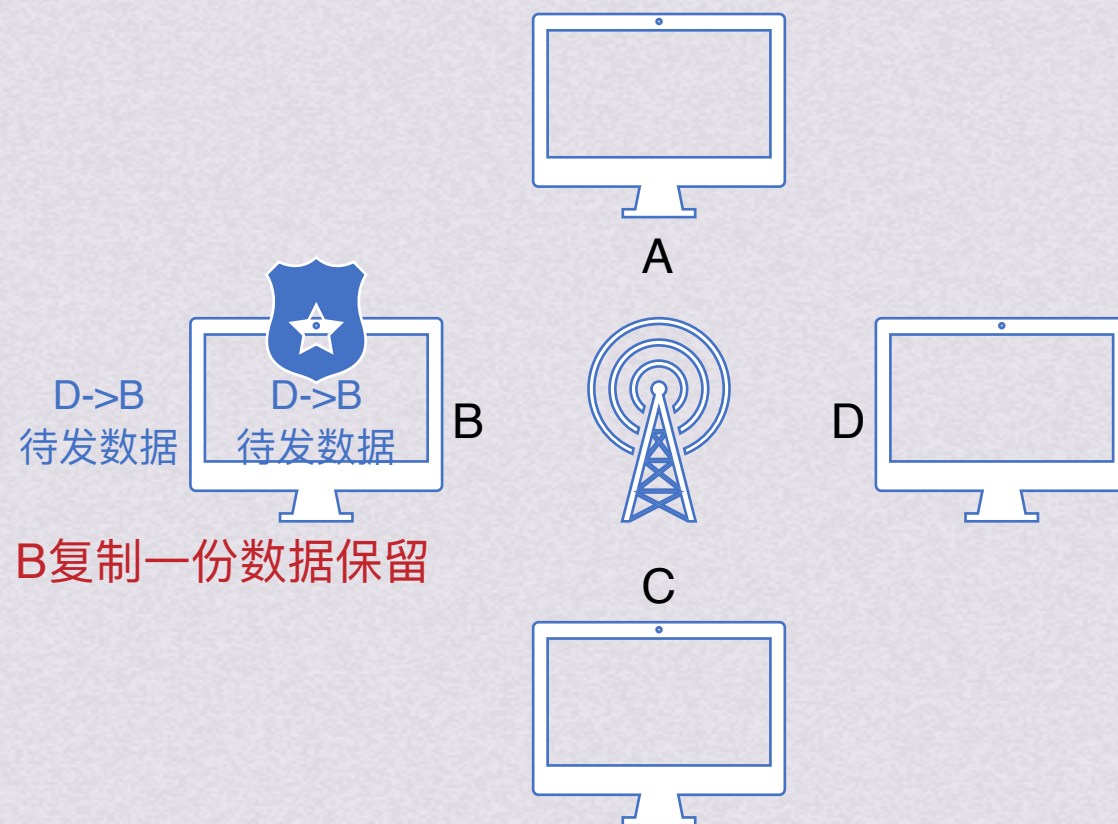


受控接入

令牌传递协议

轮训访问中，用户不能随机发送信息，必须经过集中控制的监控站，循环方式轮询每个节点。典型的轮训访问控制协议是令牌传递协议

令牌传递协议中，一个令牌沿着环形总线在各站之间依次传递。令牌是一个特殊的控制帧，本身不含信息，只控制信道使用，确保同一时刻只有一个站独占信道。站点发完一帧后，就释放令牌让其他站使用



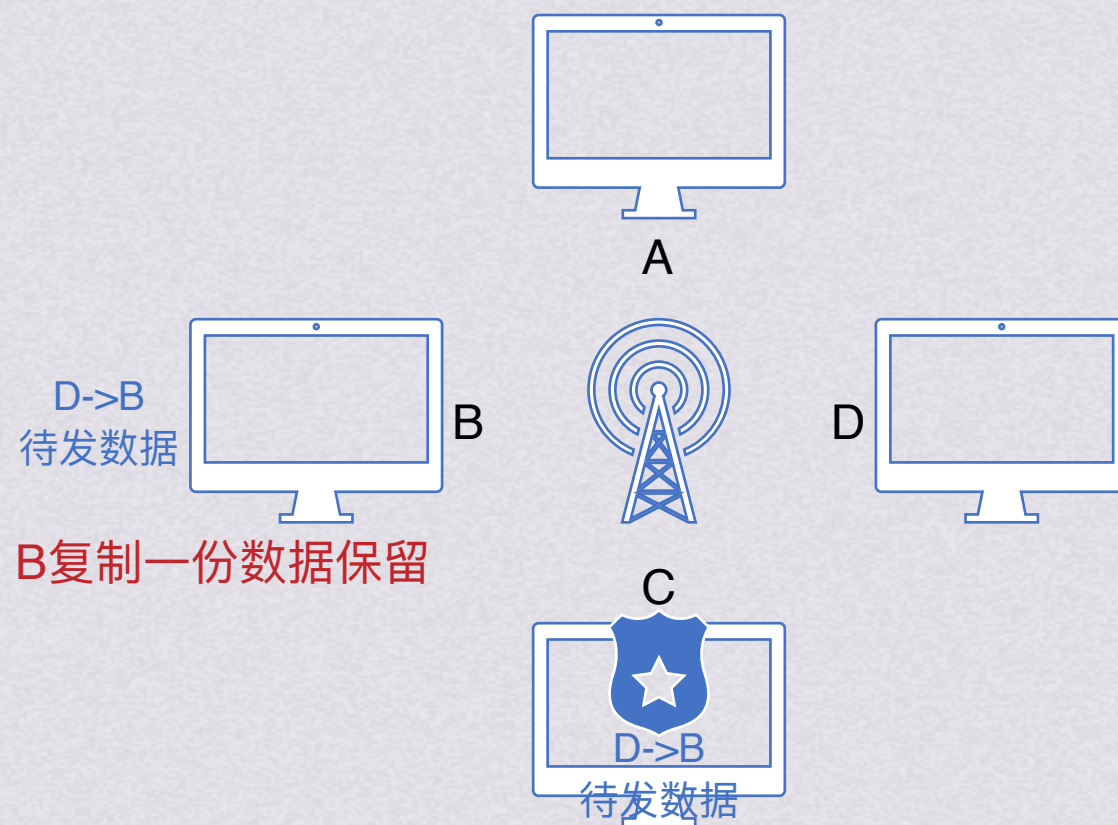


受控接入

令牌传递协议

轮训访问中，用户不能随机发送信息，必须经过集中控制的监控站，循环方式轮询每个节点。典型的轮训访问控制协议是令牌传递协议

令牌传递协议中，一个令牌沿着环形总线在各站之间依次传递。令牌是一个特殊的控制帧，本身不含信息，只控制信道使用，确保同一时刻只有一个站独占信道。站点发完一帧后，就释放令牌让其他站使用



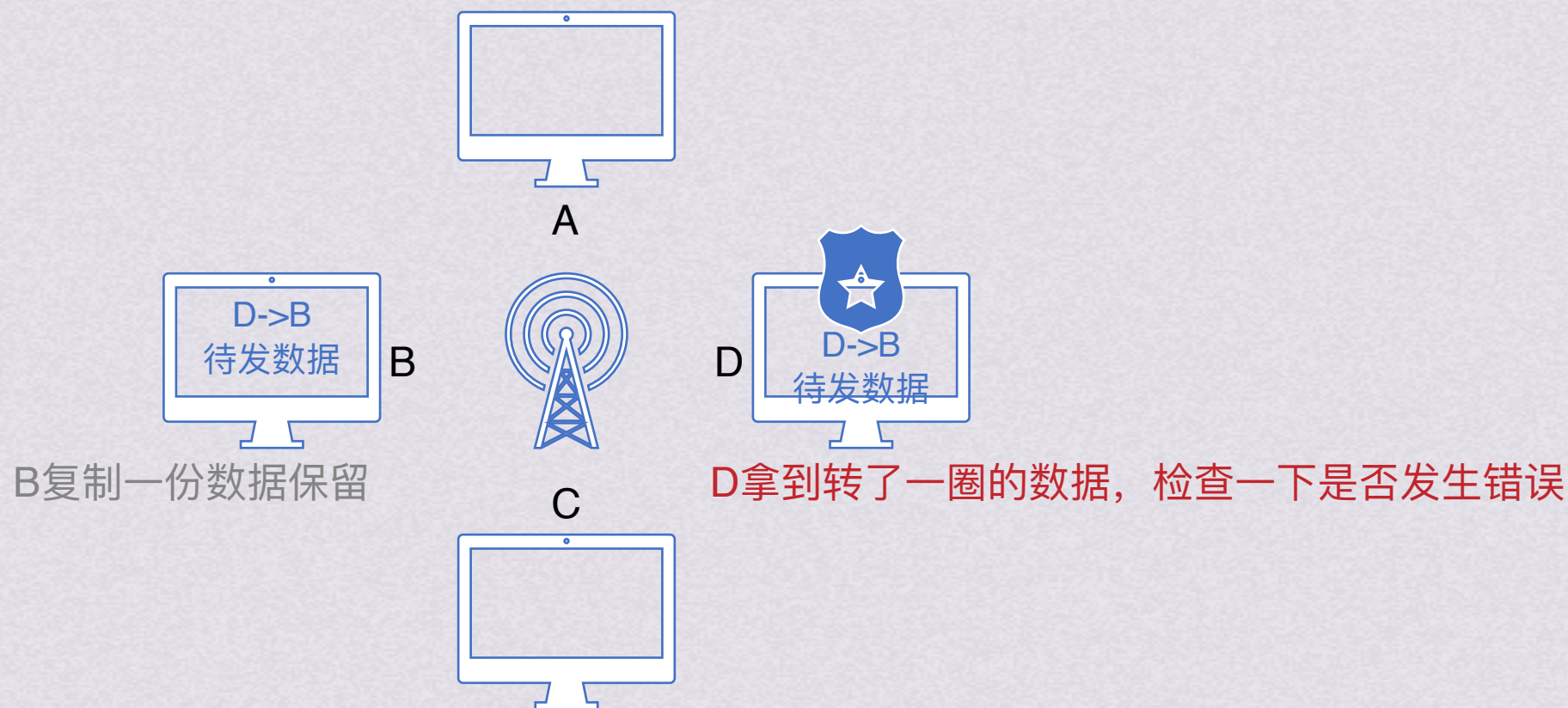


受控接入

令牌传递协议

轮训访问中，用户不能随机发送信息，必须经过集中控制的监控站，循环方式轮询每个节点。典型的轮训访问控制协议是令牌传递协议

令牌传递协议中，一个令牌沿着环形总线在各站之间依次传递。令牌是一个特殊的控制帧，本身不含信息，只控制信道使用，确保同一时刻只有一个站独占信道。站点发完一帧后，就释放令牌让其他站使用





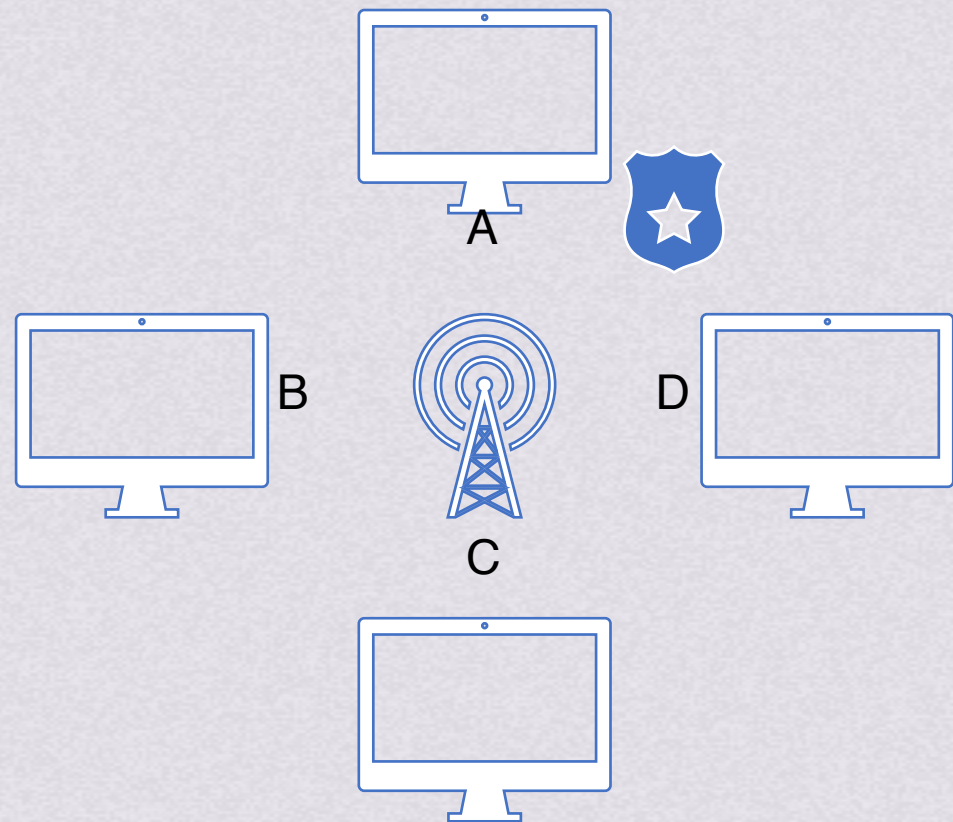
受控接入

令牌传递协议

轮训访问中，用户不能随机发送信息，必须经过集中控制的监控站，循环方式轮询每个节点。典型的轮训访问控制协议是令牌传递协议

令牌传递协议中，一个令牌沿着环形总线在各站之间依次传递。令牌是一个特殊的控制帧，本身不含信息，只控制信道使用，确保同一时刻只有一个站独占信道。站点发完一帧后，就释放令牌让其他站使用

令牌环网络中令牌和数据的传递过程如下：



1.网络空闲时，环路中只有令牌帧循环传递

2.令牌传递到有数据要发送的站点时，站点修改令牌中一个标志位，并在令牌中附加自己需要传输数据，把令牌变成数据帧发出

3.数据帧沿着环路传输，接收到的站点一边转发数据，一边查看帧的目的地址。若目的地址和自己的地址相同，则接收站复制数据帧

4.数据帧沿着环路传输，直到到达该帧的源站点，源站点收到自己发出去的帧后不再转发。同时，通过校验返回来的帧看看数据传输过程中有无错误，有则重传

5.源站点传送完数据后，重新产生一个令牌，传递给下一个站点，交出信道控制权